

Python编程基础

北京航空航天大学计算机学院

School of Computer Science and Engineering, Beihang University

黄 迪、张永飞、陈佳鑫

2026年春季学期

Spring 2026

内容大纲

● Python编程语言基础

- 变量及基本数据类型（数字、字符串、列表、字典、元组、集合）
- 输入输出流（标准输入输出、文件输入输出）
- 控制流语句（条件控制语句、循环控制语句）

● 科学计算及可视化工具

- 数值计算基础库: NumPy
- 科学计算库: SciPy
- 数据分析库: Pandas
- 绘图工具库: Matplotlib

1.1 变量及基本数据类型

变量

● 赋值语句

- 用于创建一个自定义名字的变量

变量名 = 变量值

- 第一次为变量赋值时，Python会创建变量；后续的变量赋值，会改变原有变量的绑定关系

● 变量

- 在Python中，**变量是内存中存储数据的容器**，其通过赋值操作进行定义；**变量名是一个标识符**，指向内存中的某个数据类型

注意，与C语言不同的是，Python在创建变量时**无需定义变量的数据类型**，且其类型可以通过基础类型转换操作随时进行改变

基本数据类型简介

● 基本数据类型

■ 在Python中，所谓变量的"数据类型"实际上指的是该变量所绑定的存储在内存中的对象的数据类型

■ 基本数据类型包含：

- 数字：整数int、浮点数float、复数complex、布尔数bool等
- 字符串：str
- 容器：列表list、字典dict、元组tuple、集合set等

其中，数字和字符串属于**基础类型**（即Python本身提供的，用于表示最基本的数据类型），而容器属于**引用类型**（即在Python中能够被引用的数据结构，其在内存中占用一块连续的空间，且可以被多个变量引用）

数字简介

● 数字类型

- 整数 (int) : 不带小数点的正数或负数
- 浮点数 (float) : 带有小数点的数字
- 复数 (complex) : 包含实部和虚部的数字
- 布尔数 (bool) : 只有两个值True和False

整数变量 (二进制整数、二进制整数、八进制整数、十六进制整数)

a, b, c, d = 100, 0b1101, 0o777, 0xA1B2C34D

浮点数变量 (小数写法, 科学计数法)

a, b = 3.14, 6.18E-1

复数变量 (一般虚数, 纯虚数)

a, b = 1 + 2j, -3j

布尔数变量 (真, 假)

a, b = True, False

字符串简介

● 字符串类型

■ 字符串（string）：是由字符（char）组成的序列，用单引号、双引号或三引号（三个单引号或三个双引号）括起来

字符串

```
str1 = "Hello, World !"
```

```
str2 = "My name is 'Tony' !"
```

```
str3 = 'Nice to meet you !'
```

```
str4 = """Thank you !"""
```

其中，三引号定义法与多行注释的写法一样，且同样支持换行操作，若使用变量接收它，则作为一个字符串使用；若不使用变量接收它，则作为一个多行注释使用

字符串的基本操作

● 字符串的引号嵌套

- 若使用单引号定义法，则可以内含双引号；若使用双引号定义法，则可以内含单引号；
- 可以使用反斜杠（\）转义特殊字符来解除引号效用，使之成为普通字符

```
# 字符串中的引号嵌套  
str1 = "Python" # "Python"  
str2 = 'Best one' # 'Best one'  
str3 = '\Hello, World !\' # 'Hello, World !'  
str4 = "\"Hello, World !\"" # "Hello, World !"
```


字符串的基本操作

● 字符串的拼接与重复

- 使用加号 (+) 连接字符串或字符串变量，不能与非字符串类型对象拼接；使用乘号 (*) 实现字符串复制

```
# 字符串中的拼接与重复
str1 = "Hello"
str2 = "Tony"
print(str1 + " , " + "I am " + str2 + " !") # Hello, I am Tony !
print( str1 * 2 ) # HelloHello
```

字符串的基本操作

● 字符串格式化

■ 字符串格式化基本语法: **"%占位符"%变量**, 其中%表示需要进行占位操作, 占位符表示将变量转化成为对应数据类型并标明具体位置

➤ 字符串占位符 (%s) : 变为字符串, 并放入占位的位置

```
# 字符串占位符
str1 = "Python"
print("Class name is: %s. " % str1) # Class name is: Python.

class_num = 6
grade_num = 2
print("I am in Class %s, Grade %s. " % (class_num, grade_num))
# I am in Class 6, Grade 2.
```

当存在多个变量占位时, 变量需要用括号括起来, 并按照占位的顺序填入

字符串的基本操作

● 字符串格式化

- 整数占位符 (%d) : 变为整数, 并放入占位的位置
- 浮点数占位符 (%f) : 变为浮点数, 并放入占位的位置

整数占位符

a = 111

print(" The output is: %d. " % a) # The output is 111.

print(" The output is: %5d. " % a) # The output is 111.

浮点数占位符

b = 11.345

print(" The output is: %f. " % b) # The output is 11.345000.

print(" The output is: %.2f. " % b) # The output is 11.35.

print(" The output is: %7.2f. " % b) # The output is 11.35.

精度控制"**m.n**", 其中m用于控制宽度, 要求为数字, 且设置的宽度不得小于变量自身;

.n用于控制小数点精度, 要求为数字, 会对浮点数进行小数部分的四舍五入

字符串的基本操作

● 字符串格式化

■ 格式化的快速写法: `f"内容{变量}"`

```
# 快速格式化  
str1 = " Python "  
print(f" Class name is: {str1}. ") # Class name is Python.
```

■ 表达式格式化: 对一条具有明确执行结果的代码语句进行格式化

```
# 表达式格式化  
print("The result of 1 * 2 is: %d." % (1 * 2)) # The result of 1 * 2 is: 2.  
print(f"The result of 1 * 2 is: {1 * 2}.") # The result of 1 * 2 is: 2.
```

字符串的常用操作

● 字符串的常用操作

- 查找特定字符串的下标索引值: `.index(str)`
- 字符串的替换: `.replace(str1, str2)`
- 字符串的分割: `.split`
- 字符串的规整（去前后空格）: `.strip()`
- 字符串的规整（去前后指定字符串）: `.strip(str)`
- 统计字符串中某字符串出现的次数: `.count(str)`
- 统计字符串的长度: `len(str)`
- 字符串的首字母大写: `.title()`
- 字符串的所有字母大/小写: `.upper()/lower()`

字符串的常用操作

● 字符串的常用操作

字符串的常用操作

```
str1 = "123hello World321"
```

```
print(str1.index("World")) # 10
```

```
print(str1.replace("World", "Python")) # 123hello Python321
```

```
print(str1.split(" ")) # ['123hello', 'World321']
```

```
print(str1.strip()) # 123hello World321
```

```
print(str1.strip("12")) # 3hello World3
```

```
print(str1.count("o")) # 2
```

```
print(len(str1)) # 19
```

```
print(str1.title()) # 123Hello World321
```

```
print(str1.upper()) # 123HELLO WORLD321
```

```
print(str1.lower()) # 123hello world321
```

字符串的常用操作

● 字符串中的转义字符

■ 是Python中的一种特殊字符，其通常由反斜杠（\）开始，后面跟着一个或多个字符，用于表示那些在字符串中具有特殊含义的字符或序列

转义字符	含义
\n	换行符
\t	制表符
\r	回车
\'(\")	单（双）引号
\uhhhh	4位十六进制数对应的字符
\\	一个\
\ooo	3位八进制数对应的字符
\xhh	2位十六进制数对应的字符

容器简介

● 容器类型

- 定义：是一种可以容纳多份数据的基本数据类型，每一份数据被称为一个元素，每个元素可以是任意类型的数据
- 列表（list）：由一系列按特定顺序排列的元素组成
- 元组（tuple）：由一系列不可变的且按特定顺序排列的元素组成
- 字典（dict）：由一系列键-值对组成，每个键都与一个值相关联
- 集合（set）：由一系列无序的且不重复的元素组成

其中，列表、字典和集合属于可变数据类型，即可以在不改变其标识的情况下更改存储在变量中的元素值；而元组属于不可变数据类型，变量一旦创建就不可被修改

列表简介

● 列表

- 列表 (list) 是一种内置的数据结构，用于存储一系列**按特定顺序排列**的元素，此类元素可以是不同类型的数据，如数字、字符串，甚至是其他列表（即嵌套列表）
- 列表的长度和元素是**可变**的，因此能够在创建后对其进行添加、删除或更改等操作
- 在Python中，用方括号 ([]) 来表示列表，并用逗号 (,) 分隔其中的所有元素

```
列表名 = [元素值_1, 元素值_2, 元素值_3]
```

列表的基本操作

● 列表的定义

■ 定义空列表

```
# 定义空列表  
列表名 = [] # list_1 = []  
列表名 = list() # list_2 = list()
```

■ 定义非空列表

```
# 定义非空列表  
列表名 = [元素值1, 元素值2, 元素值3] # list_1 = [1, 'a', 3.14]  
列表名 = list(一个可迭代的序列) # list_2 = list(range(10))
```

列表的基本操作

● 列表的嵌套

- 列表的嵌套也被称作二维列表（或多维列表），具体表现为某一列表内的元素均为列表

```
列表名 = [列表_1, 列表_2, 列表_3]
```

■ 定义空二维列表

定义空二维列表

```
列表名 = [空列表1, 空列表2, 空列表3] # list_1 = [[], [], []]
```

```
列表名 = list([list(), list(), list()]) # list_2 = list([list(), list(), list()])
```

列表的基本操作

● 列表的索引

■ Python为列表、元组、字符串等序列开辟了两种索引方式，分别为正向索引（从左至右）与负向索引（从右至左）：

- 正向索引：对于列表中的各个元素，依照从前向后的方向进行编号，索引从0开始，且依次递增
- 负向索引：对于列表中的各个元素，依照从后向前的方向进行编号，索引从-1开始，且依次递减

对于正向索引：



列表名 = [元素值_1, 元素值_2, 元素值_3, 元素值_4, 元素值_5]

对于负向索引：



列表名 = [元素值_-5, 元素值_-4, 元素值_-3, 元素值_-2, 元素值_-1]

列表的基本操作

● 列表的索引

■ 列表索引示例如下：

```
# 定义非空列表
name_list = ['Bob', 'Tom', 'Mike', 'Lily', 'Jenny']
# 01 访问列表中第一个元素
print(name_list[0]) # Bob
print(name_list[-5]) # Bob
# 02 访问列表中第三个元素
print(name_list[2]) # Mike
print(name_list[-3]) # Mike
# 03 访问列表中最后一个元素
print(name_list[4]) # Jenny
print(name_list[-1]) # Jenny
```

列表的基本操作

● 列表的切片

- 切片是指对列表中某一段相关的元素进行处理，即一次处理列表中的若干个元素
- 若要创建切片，可指定要使用的第一个元素的索引和最后一个元素的索引加1，以及索引的步长（默认为1）

```
name_list = ['Bob', 'Tom', 'Mike', 'Lily', 'Jenny']  
# 列表切片（输出仍为列表形式）  
print(name_list[0:3]) # 索引0~2 步长1 ['Bob', 'Tom', 'Mike']  
print(name_list[: -1]) # 索引0~3 步长1 ['Bob', 'Tom', 'Mike', 'Lily']  
print(name_list[2:]) # 索引2~4 步长1 ['Mike', 'Lily', 'Jenny']  
print(name_list[:]) # 索引0~4 步长1 ['Bob', 'Tom', 'Mike', 'Lily', 'Jenny']  
print(name_list[0: -1: 2]) # 索引0~3 步长2 ['Bob', 'Mike']  
print(name_list[4: 0: -1]) # 索引1~4 步长-1 ['Jenny', 'Lily', 'Mike', 'Tom']
```

列表的基本操作

● 列表与字符串的转化

■ 列表转字符串是指将列表或列表中元素转化为字符串形式

➤ 列表转化：使用内置函数`str()`将列表直接转化为字符

➤ 列表元素转化：使用`.join()`方法对列表中元素进行拼接并转化为字符串，此时列表元素仅能为字符串

■ 字符串转列表是指将字符串逐字符转化为列表中元素值，通常使用内置函数`list()`实现

```
list_1 = ["a", "b", "c", "d", "e"]  
# 列表转化为字符串  
str_1 = str(list_1) # ['a', 'b', 'c', 'd', 'e']  
print(str_1[0:4]) # ['a']  
# 列表元素转化为字符串  
str_2 = "".join(list_1) # abcde
```

```
str_1 = "abcde"  
# 字符串转化为列表  
list_1 = list(str_1) # ['a', 'b', 'c', 'd', 'e']  
print(list_1[0:2]) # ['a', 'b']
```

列表的基本操作

● 列表元素的添加

- 添加单个元素至列表结尾: `.append(ele)`
- 添加多个元素至列表结尾: `.extend(ele_1, ele_2)`
- 添加单个元素至指定位置: `.insert(index, ele)`

```
list_1 = ['a', 'b', 'c'] # 定义非空列表  
list_1.append('d') # append添加元素 ['a', 'b', 'c', 'd']  
list_1.extend('e', 'f') # extend添加元素 ['a', 'b', 'c', 'd', 'e', 'f']  
list_1.insert(0, 1) # insert添加元素 [1, 'a', 'b', 'c', 'd', 'e', 'f']
```

● 列表元素的修改

- 采用赋值的方式对列表元素进行修改: `list[index] = ele`

```
list_1 = ['a', 'b', 'c'] # 定义非空列表  
list_1[0] = 'd' # 赋值修改元素 ['d', 'b', 'c']
```


列表的基本操作

● 列表元素的删除

- 删除列表指定位置元素，且不返回值：del list[index]
- 删除列表结尾单个元素，且返回值：.pop()
- 删除列表指定位置元素，且返回值：.pop(index)
- 删除列表指定元素值元素：.remove(ele)
- 删除列表所有元素：.clear()

```
list_1 = ['a', 'b', 'c', 'd', 'e', 'f'] # 定义非空列表
del list_1[0] # del删除元素['b', 'c', 'd', 'e', 'f']
ele = list_1.pop() # pop删除元素['b', 'c', 'd', 'e']
ele = list_1.pop(1) # pop删除元素['b', 'd', 'e']
list_1.remove('d') # remove删除元素['b', 'e']
list_1.clear() # clear删除元素[]
```

列表的基本操作

● 列表元素的查找

- 查找指定位置元素：list[index]
- 查找指定元素在列表中的频率：.count(ele)
- 查找指定元素在列表中的索引值：.index(ele)
- 查找列表的元素数量：len(list)

```
list_1 = ['a', 'b', 'c', 'd', 'a'] # 定义非空列表  
ele = list_1[0] # 查找指定位置元素 a  
count = list_1.count('a') # count 查找指定元素频率 2  
index = list_1.index('b') # index 查找指定元素索引 1  
length = len(list_1) # len 查找元素数量 5
```

列表的基本操作

● 列表的排序与翻转

■ Python使用.sort()或.sorted()方法对列表进行排序，默认进行顺序排序

- .sort(): 对原列表进行永久性排序，没有返回值
- sorted(): 对原列表进行临时排序，有返回值

```
list_1 = [7, 4, 9, 88, 1, 5.6] # 定义非空列表
# sorted排序
sorted_list = sorted(list_1) # [1, 4, 5.6, 7, 9, 88]
print(list_1) # [7, 4, 9, 88, 1, 5.6]
# sort排序
list_1.sort()
print(list_1) # [1, 4, 5.6, 7, 9, 88]
```

若需要进行逆序排序，则在sort()或sorted()方法中引入"reverse"参数值，当reverse=True时，进行逆序排序；若reverse=False时，进行顺序排序

列表的基本操作

● 列表的排序与翻转

■ Python使用.reverse() 方法对列表进行永久性翻转，且没有返回值

```
list_1 = [7, 4, 9, 88, 1, 5.6] # 定义非空列表  
# reverse 翻转  
list_1.reverse() # [5.6, 1, 88, 9, 4, 7]  
# reverse 再翻转  
list_1.reverse() # [7, 4, 9, 88, 1, 5.6]
```

若想恢复到原本的排列顺序，只需对翻转后的列表在此调用.reverse()即可

● 列表的复制

■ Python使用.copy()方法对列表进行复制，并生成一个新的与原列表中元素完全相同的列表，但两个列表的存储地址不同，即两个列表并不指向同一列表

字典简介

● 字典

- 字典 (dictionary) 是一种内置的数据结构，用于存储一系列的元素，此类元素可以是不同类型的数据，如数字、字符串等类型，同时各元素中的**值都具有与之对应唯一的键**
- 字典中的**键是不可修改类型**，而**值可以是可修改类型**，因此能够在创建后对其进行添加、删除或更改等操作
- 在Python中，用大括号 ({ }) 来表示字典，键和值之间用冒号 (:) 分隔，而键-值对用逗号 (,) 分隔

```
字典名 = {键_1: 值_1, 键_2: 值_2, 键_3: 值_3}
```

字典的基本操作

● 字典的定义

■ 定义空字典

```
# 定义空字典  
字典名 = {} # dict_1 = {}  
字典名 = dict() # dict_2 = dict()
```

■ 定义非空字典

```
# 定义非空字典  
字典名 = {键1: 值1, 键2: 值2, 键3: 值3} # dict_1 = {'name': 'Tom', 'age': 18, 'age': 20}  
字典名 = dict(键1=值1, 键2=值2, 键3=值3) # dict_2 = dict('name' = 'Tom', 'age' = 18)
```

注意，如果创建非空字典时候重复传入相同的键，则字典最终会采用最后出现的重复键的键值对

字典的基本操作

● 字典的键和值

■ 字典是以键值对 (key : value) 为元素构成的集合，可以看作是一种特殊的集合

- 键值对是通过键来标识的，要求唯一且不重复
- 键值对的排列是无序的，无法通过索引访问键值对
- 键要求必须**唯一**，是不可变数据类型，且一旦创建就不允许修改
- 值可以是任意可变的数据类型，且可取重复值

因此，在字典中，能够根据唯一的"键"索引到具体的"值"，而不能够根据可重复的"值"索引到所对应的"键"

字典的基本操作

● 字典元素的提取

■ 作为映射类型数据结构，字典并不存在索引概念，且不支持切片操作等处理方法，因此需要利用键和值对应的映射关系实现元素提取，具体格式为：**字典名[键]**

```
# 定义非空字典
```

```
my_dict = {'name': 'Tom', 'gender': 'male', 'age': 20, 'height': 180}  
print(my_dict['name']) # 'name'键所映射的值为'Tom'
```

■ 为避免字典元素提取时传入的键不存在而导致出现异常：

➤ 使用in语句测试键是否存在于字典中

```
my_dict = {'name': 'Tom', 'gender': 'male', 'age': 20, 'height': 180}  
my_dict['height'] # 'height'键存在，则为True  
my_dict['weight'] # 'weight'键不存在，则为False
```


字典的基本操作

● 字典元素的提取

- 使用.get()方法测试键是否存在于字典中：1) 当键存在于字典时，直接返回其对应的元素值；2) 当键不存在于字典时，直接返回None；3) 当键不存在于字典，且在方法中传入代替值时，直接返回代替值而不返回None

```
# 定义非空字典
my_dict = {'name': 'Tom', 'gender': 'male', 'age': 20, 'height': 180}
# 01 'name'存在于字典，返回'Tom'
my_dict.get('name')
# 02 'weight'不存在于字典，返回None
my_dict.get('weight')
# 03 'weight'不存在于字典且有代替值，返回'Not in this dict.'
my_dict.get('weight', 'Not in this dict.')
```

字典的基本操作

● 字典元素的添加

- 添加单个元素至字典中: `dict[key] = value`
- 合并两字典中的元素至同一字典中: `.update(new_dict)`

```
dict_1 = {'a': 1, 'b': 2} # 定义非空字典dict_1  
dict_1['c'] = 3 # 复制添加元素{'a': 1, 'b': 2, 'c': 3}  
dict_2 = {'d': 4, 'a': 0} # 定义非空字典dict_2  
dict_1.update(dict_2) # update添加元素{'a': 0, 'b': 2, 'c': 3, 'd': 4}
```

● 字典元素的修改

- 采用赋值方式对字典元素进行修改: `dict[key] = value`

```
dict_1 = {'a': 1, 'b': 2, 'c': 3} # 定义非空字典  
dict_1['a'] = 0 # 赋值修改元素{'a': 0, 'b': 2, 'c': 3}
```

字典的基本操作

● 字典元素的删除

- 删除字典指定键值元素，且不返回值：del dict[key]
- 删除字典指定键值元素，且返回值：.pop(key)
- 删除字典所有元素：.clear()
- 删除字典：del dict

```
dict_1 = {'a': 1, 'b': 2, 'c': 3, 'd': 4} # 定义非空字典
del dict_1['b'] # del删除元素{'a': 1, 'c': 3, 'd': 4}
value = dict_1.pop('c') # pop删除元素{'a': 1, 'd': 4}
dict_1.clear() # clear删除元素{}
del dict_1 # del删除字典
```

字典的基本操作

● 字典元素的查找

- 查找指定键值元素: `dict[key]`
- 查找字典中的所有键: `.keys()`
- 查找字典中的所有值: `.values()`
- 查找字典中的所有键值对: `.items()`
- 查找字典的元素数量: `len(dict)`

`.keys()`、`.values()`、`.items()`方法所返回的结果是字典中所有键、值以及键值对的迭代形式，可以通过`list()`函数将返回结果转换为列表类型

```
dict_1 = {'a': 1, 'b': 2, 'c': 3, 'd': 4} # 定义非空字典
value = dict_1['a'] # 查找指定键值元素 1
keys = list(dict_1.keys()) # keys 查找字典中的所有键 ['a', 'b', 'c', 'd']
values = list(dict_1.values()) # values 查找字典中的所有值 [1, 2, 3, 4]
items = list(dict_1.items()) # items 查找字典中的所有键值对 [('a', 1), ('b', 2), ('c', 3), ('d', 4)]
length = len(dict_1) # len 查找元素数量 4
```

元组简介

● 元组

- 元组 (tuple) 是一种内置的数据结构，用于存储一系列**不可变的且按特定顺序排列**的元素，此类元素可以是不同类型的数据，如数字、字符串等类型
- 元组的长度和元素是**不可变**的，因此不能够在创建后对其进行添加、删除或更改等操作
- 在Python中，用小括号 (()) 来表示列表，并用逗号 (,) 分隔其中的所有元素

```
元组名 = (元素值_1, 元素值_2, 元素值_3)
```

元组的基本操作

● 元组的定义

■ 定义空元组

```
# 定义空元组  
元组名 = () # tuple_1 = ()  
元组名 = tuple() # tuple_2 = tuple()
```

■ 定义非空元组

```
# 定义非空元组  
元组名 = (元素值1, 元素值2, 元素值3) # tuple_1 = (1, 'a', 3.14)  
元组名 = tuple(一个可迭代的序列) # tuple_2 = tuple(range(10))
```

● 元组的相关操作

■ 元组无法进行增删改操作，查找操作与列表查找操作相同

集合简介

● 集合

- 集合 (set) 是一种内置的数据结构，用于存储一系列**无序的且不重复**的元素，此类元素可以是不同不可变类型的数据，如数字、字符串、元组等类型
- 集合是**可修改类型**，因此能够在创建后对其进行添加、删除或更改等操作
- 在Python中，用大括号 ({ }) 来表示集合，并用逗号 (,) 分隔其中的所有元素

集合名 = {元素值_1, 元素值_2, 元素值_3}

元素值类型不可变!

集合的基本操作

● 集合的定义

■ 按照数据结构对象是否可变进行分类，集合数据类型包括可变集合类型和不可变集合类型

➤ 可变集合：集合是可变对象，可对其进行元素的添加、删除、修改等处理，处理结果直接作用于集合对象上

```
# 定义空可变集合
```

```
集合名 = set() # set_1 = set()
```

```
# 定义非空可变集合
```

```
集合名 = {元素值1, 元素值2, 元素值3} # set_1 = {'a', 'b', 'c', 'a', 'd'}
```

```
列表名 = [元素值1, 元素值2, 元素值3] # list_1 = ['a', 'b', 'c', 'a', 'd']
```

```
集合名 = set(列表名) # set_2 = set(list_1) set()函数将列表转换为集合
```

注意，定义空集合只能采用set()函数，而用集合名 = {}的方式仅能定义空字典；如果创建非空集合时候重复传入相同的元素值，则集合最终会保留一个重复出现的元素值

集合的基本操作

● 集合的定义

- 不可变集合：集合是不可变对象，不可对其进行修改处理，使用frozenset()函数定义不可变集合

```
# 定义空不可变集合
```

```
集合名 = frozenset() # set_1 = frozenset()
```

```
# 定义非空不可变集合
```

```
列表名 = [元素值1, 元素值2, 元素值3] # list_1 = ['a', 'b', 'c', True, 1]
```

```
集合名 = frozenset(列表名) # set_2 = frozenset(list_1)
```

接下来，仅对可变集合的相关操作和处理进行介绍

集合的基本操作

● 集合元素的提取

- 由于集合内元素无序，因此不能以索引或下标的方式访问单个元素，且无法直接修改集合中的单个元素值，只能以整体打印输出的方式查看集合元素值

```
# 定义非空集合  
set_1 = {'a', 'b', 'c', 'd', 'a'}  
print(set_1) # {'a', 'b', 'c', 'd'}
```

● 集合元素的添加

- 添加单个元素至集合中： `.add(ele)`

```
# 定义非空集合  
set_1 = {'a', 'b', 'c', 'd'}  
set_1.add('e') # {'a', 'b', 'c', 'd', 'e'}
```

集合的基本操作

● 集合元素的删除

- 删除集合指定元素，且会抛出异常：.remove(ele)
- 删除集合指定元素，且不会抛出异常：.discard(ele)
- 删除集合随机一个元素：.pop()
- 删除字典中所有元素：.clear()

```
# 定义非空集合
set_1 = {'a', 'b', 'c', 'd'}
set_1.remove('e') # remove 删除元素 KeyError
set_1.remove('b') # remove 删除元素 {'a', 'c', 'd'}
set_1.discard('e') # discard 删除元素 {'a', 'c', 'd'}
set_1.pop() # pop 删除元素 {'a', 'd'}
set_1.clear() # clear 删除元素 {}
```

集合基本操作

● 集合元素的修改

- 修改一个集合为两个集合的并集: `.update(new_set)`
- 修改一个集合为与另一个集合的差集: `.difference_update(new_dict)`

```
set_1 = set_2 = {'a', 'b', 'c'} # 定义非空集合 set_1、set_2  
set_3 = {'a', 'd', 'e'} # 定义非空集合 set_3  
set_1.update(set_3) # 修改集合为并集 {'a', 'b', 'c', 'd', 'e'}  
set_2.difference_update(set_3) # 修改集合为差集 {'b', 'c'}
```

● 集合元素的查找

- 查找集合的元素数量: `len(set)`

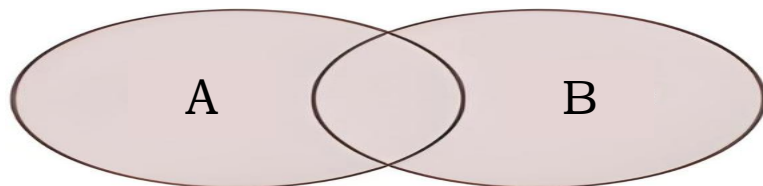
```
set_1 = {'a', 'b', 'c'} # 定义非空集合  
length = len(set_1) # len 查找元素数量 3
```

集合的运算

● 集合的运算

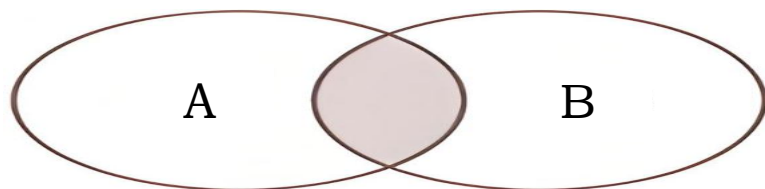
■ 对于两个集合A与B，则有：

- 并集（union），即由属于A或B的所有元素组成的集合，称为A与B的并集，记为 $A \cup B = \{x|x \in A \text{ or } x \in B\}$



```
set_1 = {'a', 'b', 'c'} # 定义非空集合 set_1  
set_2 = {'a', 'd', 'e'} # 定义非空集合 set_2  
set_1.union(set_2) # 并集{'a', 'b', 'c', 'd', 'e'}
```

- 交集（intersection），即由同时属于A和B的元素组成的集合，称为A与B的交集，记为 $A \cap B = \{x|x \in A \text{ and } x \in B\}$

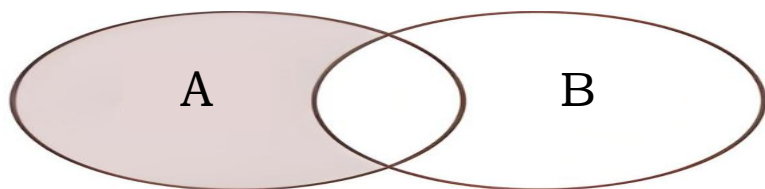


```
set_1 = {'a', 'b', 'c'} # 定义非空集合 set_1  
set_2 = {'a', 'd', 'e'} # 定义非空集合 set_2  
set_1.intersection(set_2) # 交集{'a'}
```

集合的运算

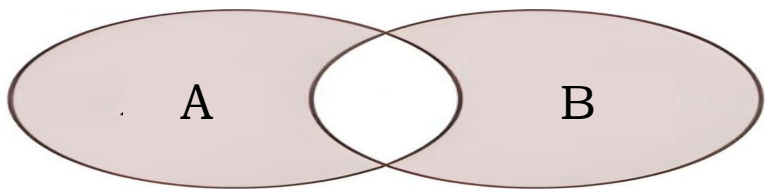
● 集合的运算

- 差集 (difference) , 即由属于A而不属于B的元素组成的集合, 称为A与B的差集, 记为 $A - B = \{x | x \in A \text{ and } x \notin B\}$



```
set_1 = {'a', 'b', 'c'} # 定义非空集合 set_1  
set_2 = {'a', 'd', 'e'} # 定义非空集合 set_2  
set_1.difference(set_2) # 差集 {'b', 'c'}
```

- 异或集 (symmetric_difference) , 即由不同时属于A和B的元素组成的集合, 称为A与B的异或集, 记为 $A \wedge B = (A \cup B) - (A \cap B)$



```
set_1 = {'a', 'b', 'c'} # 定义非空集合 set_1  
set_2 = {'a', 'd', 'e'} # 定义非空集合 set_2  
set_1.symmetric_difference(set_2) # 异或集  
'b', 'c', 'd', 'e'
```

基本数据类型判断

● 基本数据类型判断

- 查询变量的基本数据类型： `type(var)`
- 判断变量是否为指定基本数据类型的实例： `isinstance(var, type)`
- 判断变量是否为指定基本数据类型： `is`

```
a, b, c, d = 20, 5.5, True, [1, 2, 3] # 定义变量
type(a), type(b) # type判断数据类型 <class 'int'> <class 'float'>
isinstance(c, int) # isinstance判断数据类型 True
type(d) is list # is判断数据类型 True
```

基本数据类型转换

● 基本数据类型转换

■ 在Python中，有时基本数据类型会自动转换，无需人工干预，如：

```
a, b = 123, 1.23 # 定义变量  
c = a + b  
type(a), type(b), type(c) # <class 'int'> <class 'float'> <class 'float'>
```

■ 在某些情况下，为了满足计算需求，需要使用内置函数强制转换数据类型，如：

```
a, b = 123, '1.23' # 定义变量  
type(a), type(b) # <class 'int'> <class 'str'> a和b无法直接进行计算  
c = a + float(b) # float()强制转换数据类型  
type(a), type(b), type(c) # <class 'int'> <class 'str'> <class 'float'>
```


基本数据类型转换

● 基本数据类型转换

函数	描述
<code>int(x, base)</code>	将对象x转换为一个整数， <code>base</code> 用于设置进制数
<code>float(x)</code>	将对象x转换为一个浮点数
<code>complex(real, imag)</code>	定义一个复数， <code>real</code> 为实部， <code>imag</code> 为虚部
<code>str(x)</code>	将对象x转换为一个字符串
<code>repr(x)</code>	将对象x转换为表达式字符串
<code>tuple(s)</code>	将序列s转化为一个元组
<code>list(s)</code>	将序列s转化为一个列表
<code>set(s)</code>	将序列s转化为一个可变集合
<code>frozenset(s)</code>	将序列s转化为一个不可变集合
<code>dict(d)</code>	定义一个字典， <code>d</code> 是一个(key, value)元组序列
<code>chr(x)</code>	返回整数x对应的ASCII字符
<code>ord(c)</code>	返回字符c对应的ASCII数值
<code>hex(x)</code>	将整数x转换为一个十六进制字符串
<code>oct(x)</code>	将整数x转换为一个八进制字符串

运算符简介

● 算数运算符：用于执行基本的数学运算

- +（加法）：将两个数相加
- -（减法）：将两个数相减
- *（乘法）：将两个数相乘
- /（除法）：将两个数相除，结果为浮点数
- //（整除）：将两个数相除，结果向下取整
- %（取余）：返回两个数相除的余数
- **（幂运算）：返回一个数的指定次幂

`a, b = 10, 3 # 定义对象`

`a + b # 13`

`a - b # 7`

`a * b # 30`

`a / b # 3.33333333335`

`a % b # 1`

`a // b # 3`

`a ** b # 1000`

运算符简介

● 赋值运算符：用于将值赋给变量

■ =（简单赋值）：将右边的值赋给左边的变量

■ +=（加法赋值）、-=（减法赋值）、*=（乘法赋值）、/=（除法赋值）、//=（整除赋值）：将左边的变量加上/（减去/乘以/除以/整除以）右边的值，并将结果赋给左边的变量

■ %=（取余赋值）：将左边的变量除以右边的值的余数赋给左边的变量

■ **=（幂运算赋值）：将左边的变量的指定次幂赋给左边的变量

```
a = 5 # 定义对象
```

```
a += 3 # 8
```

```
a -= 2 # 6
```

```
a *= 4 # 24
```

```
a /= 3 # 8.0
```

```
a //= 2 # 4.0
```

```
a %= 3 # 1.0
```

```
a **= 2 # 1.0
```

1.2 输入输出流

输入输出流

● 输入输出流

■ 在Python中，输入Input/输出Output (I/O) 是指程序与外部世界之间的数据交换，包含：

- 标准输入输出：指程序与键盘、显示器等标准设备之间的数据交换
- 文件输入输出：指程序与文件之间的数据交换

标准输入输出

● 标准输入输出

- 在Python中，通常使用input()函数获取用户输入，使用print()函数进行数据输出
- 标准输入输出通常使用以下三个文件对象：

类型	对象	描述	默认指向
标准输入	sys.stdin	从键盘输入的数据	键盘
标准输出	sys.stdout	输出到显示器	显示器
标准错误输出	sys.stderr	输出到显示器	显示器

标准输入输出

● 标准输入

■ 标准输入包含input()函数和sys.stdin模块两部分：

- input()函数：允许从键盘读取用户的输入，一般会显示一个输入提示，等待用户输入，并返回用户输入的字符串

```
# 给定提示并等待用户输入  
name = input('Enter your name: ') # 'Tom'
```

- input()函数默认返回字符串类型，若需要将输入转化为其他类型，需要显式地进行类型转化

```
# 给定提示并等待用户输入  
age = int(input('Enter your age: ')) # 20
```

标准输入输出

● 标准输入

- `sys`模块中的`stdin`对象允许从标准输入中读取数据，通常可以使用`sys.stdin.readline()`方法读取一行输入

```
# 引入sys模块
import sys
# 调用sys模块
line = sys.stdin.readline() # 读取一行输入
```


标准输入输出

● 标准输出

■ 标准输出包含output()函数和sys.stdout模块两部分：

- print()函数：是最常见的输出方式，将字符串表示的值打印至标准输出（通常是屏幕）

```
# 字符串输入
```

```
print('My name is Tom.') # My name is Tom.
```

- print()函数支持使用end参数定义输出结束时的字符（默认为换行符），或使用sep参数定义输出多个元素时的分隔符

```
print('Hello, ', end = ' ')
```

```
print('World !') # 定义end参数，输出 Hello, World !
```

```
print('Tom', 'Mike', 'Mary', sep = ', ') # 定义sep参数，输出 Tom, Mike, Mary
```

标准输入输出

● 标准输出

- `sys`模块中的`stdout`对象允许直接将数据写入标准输出中，通常可以使用`sys.stdout.write()`方法写入标准输出，适用于需要更多控制输出格式的情况

```
# 引入sys模块
import sys
# 调用sys模块
line = sys.stdout.write('Hello, World !\n') # 写入一次输出
```

文件输入输出

● 文件输入输出

操作	函数	描述
打开文件	open()	打开文件
读文件	read()	读取文件内容
写文件	write()	写入文件内容

● 打开文件

■ 在Python中，使用open()函数能够打开一个文件，并返回一个文件对象，可以对该对象进行读写操作

```
# 打开文件 (filename为文件名, mode为文件打开模式)
file_1 = open (filename, mode) #打开文件file_1
with open (filename, mode) as file_2 #使用with语句打开文件file_2, 处理结束后会自动关闭文件并释放资源
```

文件输入输出

● 打开文件

模式	解释	描述
'r'	读取模式 (Read)	打开文件用于读取。文件的指针将会放在文件的开头。这是默认模式。
'w'	写入模式 (Write)	打开文件用于写入。如果文件存在，则覆盖文件。如果文件不存在，则创建新文件。
'a'	追加模式 (Append)	打开文件用于追加内容。文件的指针将会放在文件的末尾。如果文件不存在，则创建新文件。
'rb'	二进制读取模式 (Read Binary)	以二进制格式打开一个文件用于只读。文件的指针将会放在文件的开头。这个模式通常用于非文本文件，比如图像、音频等。
'wb'	二进制写入模式 (Write Binary)	以二进制格式打开一个文件用于写入。如果文件存在，则覆盖文件。如果文件不存在，则创建新文件。这个模式也通常用于非文本文件的写入，比如图像、音频等。
'ab'	二进制追加模式 (Append Binary)	以二进制格式打开一个文件用于追加内容。文件的指针将会放在文件的末尾。如果文件不存在，则创建新文件。这个模式也通常用于非文本文件的追加，比如图像、音频等。
'r+'	读写模式 (Read/Write)	打开一个文件用于读写操作。文件的指针将会放在文件的开头。
'w+'	写读模式 (Write/Read)	打开一个文件用于读写操作。如果文件存在，则覆盖文件。如果文件不存在，则创建新文件。文件的指针将会放在文件的开头。
'a+'	追加读模式 (Append/Read)	打开一个文件用于读写操作。文件的指针将会放在文件的末尾。如果文件不存在，则创建新文件。
'rb+'	二进制读写模式 (Read/Write Binary)	以二进制格式打开一个文件用于读写操作。文件的指针将会放在文件的开头。这个模式通常用于非文本文件的读写，比如图像、音频等。
'wb+'	二进制写读模式 (Write/Read Binary)	以二进制格式打开一个文件用于读写操作。如果文件存在，则覆盖文件。如果文件不存在，则创建新文件。这个模式也通常用于非文本文件的读写，比如图像、音频等。
'ab+'	二进制追加读模式 (Append/Read Binary)	以二进制格式打开一个文件用于读写操作。文件的指针将会放在文件的末尾。如果文件不存在，则创建新文件。这个模式也通常用于非文本文件的读写，比如图像、音频等。

文件输入输出

● 读取文件

- 读取文件指定字节长度的内容（如果不指定参数，则读取文件全部内容）并以字符串形式返回：.read(size)

```
with open ('file.txt', 'r') as file : # 打开文件
    content = file.read() # read 读取文件所有内容
```

- 读取文件一行内容，包含行尾换行符（如果不指定参数，则读取文件当前行全部内容），并以字符串形式返回：.readline()

```
with open ('file.txt', 'r') as file : # 打开文件
    first_line = file.readline() # readline 读取文件第一行
    second_line = file.readline() # readline 读取文件第二行
```

文件输入输出

● 读取文件

■ 读取文件全部内容（如果不指定参数，则读取文件全部内容）并以列表形式返回，列表中每个元素为一行内容：`.readlines()`

```
with open('file.txt', 'r') as file : # 打开文件
    lines = file.readlines() # readlines 读取文件所有行
```

方法	功能	返回类型	主要用途
<code>read()</code>	读取整个文件或指定字节数	字符串	读取文件的全部内容
<code>readline()</code>	逐行读取文件	字符串	按行处理文件内容
<code>readlines()</code>	读取文件所有行并返回列表	列表	读取多行内容并存储于列表中

文件输入输出

● 写入文件

■ 写入字符串数据到文件中：.write(str)

```
with open ('file.txt', 'w') as file : # 打开文件  
    file.write('Hello, World !\n') # write 将字符串写入文件
```

■ 写入字符串列表到文件中，列表中每个元素为一行内容（需要在列表中手动添加换行符）：.writelines(list)

```
lines = ['Hello, \n', 'World !']  
with open ('file.txt', 'r') as file : # 打开文件  
    file.writelines(lines) # writelines 将字符串列表写入文件
```

1.3 控制流语句

条件控制语句

● 条件语句的基本语法

```
if condition_1:
    # 如果条件1为真，则执行这里的代码块
    statement_1
    statement_2
elif condition_2:
    # 如果条件1为假，条件2为真，则执行这里的代码块
    statement_3
    statement_4
else:
    # 如果条件1和2为假，则执行这里的代码块
    statement_5
    statement_6
```

注意，一个完整的条件控制结构中，1) 可以有若干个elif，用于测试若干个条件，若之前的if或elif条件为假，则继续测试下一个条件；2) 可以有一个可选的else语句，用于指定如果上述条件都为假时要执行的代码块

条件控制语句

● 条件测试

- 每个条件语句的核心都是一个值为True或False的表达式，这种表达式被称为**条件测试**
- 如果条件测试的值为True，则执行紧跟在当前条件语句后面的代码块；如果为False，则忽略这些代码
- 条件测试中通常包含以下运算符：
 - 比较运算符：比较两个值或变量的大小或相等性
 - 逻辑运算符：组合多个条件，并判断条件间的逻辑关系
 - 成员运算符：判断一个值是否存在于一个序列或集合中
 - 身份运算符：比较两个对象是否为同一个对象

条件控制语句

● 条件测试

■ 比较运算符

- ==（等于）：判断两个值是否相等
- !=（不等于）：判断两个值是否不相等
- >（大于）：判断左边的值是否大于右边的值
- <（小于）：判断左边的值是否小于右边的值
- >=（大于等于）：判断左边的值是否小于或等于右边的值
- <=（小于等于）：判断左边的值是否小于或等于右边的值

```
a, b = 5, 8 # 定义比较对象
```

```
a == b # False      a < b # True
```

```
a != b # True       a >= b # False
```

```
a > b # False       a <= b # True
```

条件控制语句

● 条件测试

■ 逻辑运算符

- and（与）：当所有条件均为True时，返回True；否则返回False
- or（或）：当至少有一个条件为True时，返回True；否则返回False
- not（非）：对条件取反，如果条件为True，则返回False；如果条件为False，则返回True

```
a, b = True, False # 定义比较对象  
a and b # False  
a or b # True  
not a # False
```

条件控制语句

● 条件测试

■ 成员运算符

- in: 如果值存在于序列或集合中, 则返回True; 否则返回False
- not in: 如果值不存在于序列或集合中, 则返回True; 否则返回False

```
list_1, str_1 = [1, 2, 3, 4, 5], "hello" # 定义序列
3 in list_1 # True
6 not in list_1 # True
'h' in list_1 # True
'e' not in list_1 # False
```

条件控制语句

● 条件测试

■ 身份运算符

- `is`: 如果两个对象是同一个对象, 则返回`True`; 否则返回`False`
- `is not`: 如果如果两个对象不是同一个对象, 则返回`True`; 否则返回`False`

```
list_1, list_2 = [1, 2, 3], [1, 2, 3] # 定义比较对象
list_3 = list_1
list_1 is list_3 # True
list_1 is list_2 # False
list_2 is not list_3 # True
```

注意, `is`比较的是两个对象的身份, 即它们在内存中是否指向同一个地址; `==`比较的是两个对象的值是否相等

条件控制语句

● if语句

■ if语句是最基础的条件语句，其功能是依据条件判断是否执行某段代码，即如果满足某个条件，就执行相应操作

```
if condition_1: # 如果条件1为真，则执行这里的代码块
    statement_1
```

if语句

```
if condition_1:
    # 如果条件1为真，则执行这里的代码块
    statement_1
else:
    # 否则，执行这里的代码块
    statement_2
```

if-else语句

● if-else语句

■ if-else语句允许在条件成立和不成立时，分别执行不同的代码块，即如果满足某个条件，就执行一个操作；否则，执行另一个操作

条件控制语句

● if-elif-else语句

■ if-elif(else if)-else语句允许在需要判断多种条件时，分别执行不同的代码块，即依次判断各个条件，如果满足其中一个条件，就执行相应代码块；否则，继续判断条件

```
if condition_1:
    # 如果条件1为真，则执行这里的代码块
    statement_1
elif condition_2:
    # 如果条件1为假，条件2为真，则执行这里的代码块
    statement_2
else:
    # 否则，执行这里的代码块
    statement_3
```

if-elif-else
语句

条件控制语句

● 三元运算符

■ 三元运算符是条件语句中，一种简化的条件表达式，即使用一行代码，即可根据条件为真或为假执行不同的操作

result = true_value if condition else false_value

其中，condition是一个条件表达式，如果其为真，则result被赋值为true_value；否则将被赋值为false_value

```
x, y = 10, 20
```

```
# 如果 x>y, 则 x为最大值; 否则, y为最大值
```

```
max_value = x if x > y else y
```

```
print(max_value) # 20
```

循环控制语句

● 循环结构

- 条件循环（while循环）：只要条件判断为真，其包含的某条语句或某个语句块就会一直被执行，直至条件判断为假
- 取值循环（for-in循环）：通常用于遍历各类可迭代对象，如列表、元组、字符串、字典等；是一种"轮询"机制，对于可迭代对象中的每一个元素，都执行一段相同代码

while 循环

while condition:

如果条件为真，则重复执行这里的代码块

statement

for-in 循环

for var in iterable_object:

对于可迭代对象中的每个元素均重复执行这里的代码块

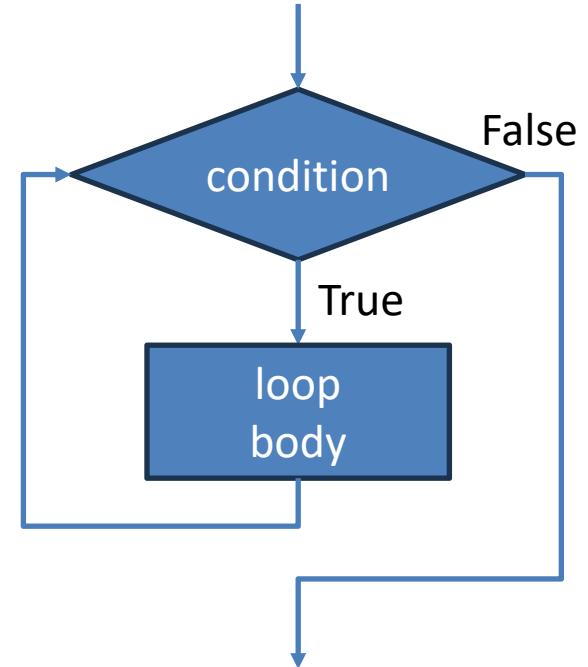
statement

循环控制语句

● while循环

■ while循环的执行流程：

- 初始化变量
- 条件判断
- 执行循环体
- 修改变量



while 循环

`while condition:`

如果条件为真，则重复执行这里的代码块

`statement`

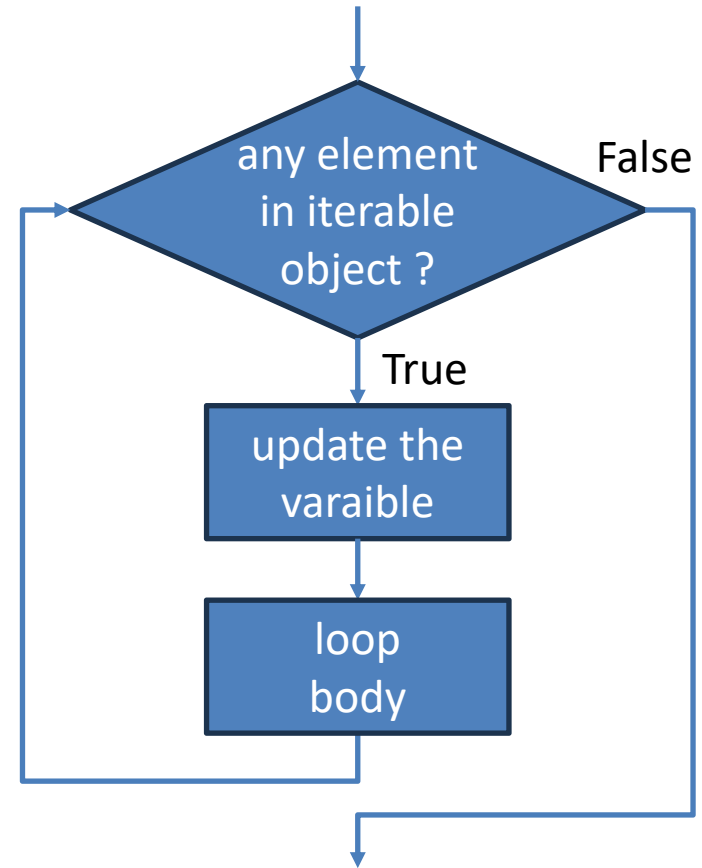
while循环结构与if条件语句的区别：
while循环对条件进行判断，如果条件为True，就一直执行循环体；
if分支对条件进行1次判断，如果条件为True，就执行1次分支代码块；

循环控制语句

● for-in循环

■ for-in循环遍历的对象必须是可迭代对象（即序列类型），其内容可以一个个依次取出，包括字符串、列表、元组、字典等

```
# for-in 循环
for var in iterable_object:
    # 对于可迭代对象中的每个
    # 元素均重复执行这里的代码块
    statement
```



循环控制语句

● 循环结构

■ 为更直观理解循环结构及其用法，在此分别使用while循环和for-in循环打印"九九乘法表"：

```
# while 循环
i = 1
while i <= 9: # 外层循环
    j = 1
    while j <= i and j <= 9: # 内层循环
        print(f"{j} * {i} = {j * i}\t", end="")
        j += 1
    print(); i += 1
```

```
# for-in 循环
for i in range(1, 10): # 外层循环
    for j in range(1, 10): # 内层循环
        if j <= i:
            print(f"{j} * {i} = {j * i}\t", end="")
    print()
```

值得注意的是，上述两类循环结构均采用了嵌套循环，即循环结构中又嵌套了另外的完整的循环结构，其中内层循环作为外层循环的循环体执行

循环控制语句

● 流程控制语句

- break语句：用于立即结束循环结构，
- continue语句：用于立即结束当前循环，并进入下次循环
- pass语句：用于占位不执行任何操作的代码块或循环体

while 循环

```
while condition_1:
    statement # 循环体
    if condition_1:
        break # 结束循环结构
    elif condition_2:
        continue # 仅结束当前循环
    else:
        pass # 不执行任何操作
```

for-in 循环

```
for var in iterable_object:
    statement # 循环体
    if condition_1:
        break # 结束循环结构
    elif condition_2:
        continue # 仅结束当前循环
    else:
        pass # 不执行任何操作
```

内容大纲

● Python编程语言基础

- 变量及基本数据类型（数字、字符串、列表、字典、元组、集合）
- 输入输出流（标准输入输出、文件输入输出）
- 控制流语句（条件控制语句、循环控制语句）

● 科学计算及可视化工具

- 数值计算库: NumPy
- 数据分析库: Pandas
- 绘图工具库: Matplotlib

2.1 数值计算库: NumPy

NumPy库

● NumPy库简介

- Python中的数值计算基础库，全称Numeric Python
- 核心对象是 ndarray，功能包括N维**同构**数组创建及运算、线性代数计算、**三角**函数计算、**多项式拟合**、傅立叶变换、生成随机数、矩阵运算等
- 是Python科学计算生态系统的基础，广泛应用于数据分析、机器学习、图像处理等领域

```
# Numpy 引入示例  
import numpy as np  
a = np.array([1, 2, 3, 4, 5])  
b = np.array([[1, 2, 3], [4, 5, 6]])
```

```
# Numpy 引入示例  
import numpy as np  
x = np.linspace(0, 10, 100)  
y = np.sin(x)
```

NumPy库常用内置方法

● 数组的创建

- 从现有数据创建数组: `np.array()`
- 指定形状的全零数组: `np.zeros()`
- 指定形状的全一数组: `np.ones()`
- 指定形状且填充特定值的数组: `np.full()`
- 给定数组起止范围, 指定步长的数组: `np.arange()`
- 给定数组起止范围, 指定个数的数组: `np.linspace()`
- 指定形状的单位矩阵或对角矩阵: `np.eye()`
- 指定形状的随机数组: `np.random.rand()`
- 符合正态分布的随机数组: `np.random.randn()`

NumPy库常用内置方法

● 数组的属性

- 形状: `arr.shape`
- 维度: `arr.ndim`
- 元素总数: `arr.size`
- 数据类型: `arr.dtype`
- 类型转换: `arr.astype()`

```
import numpy as np
arr = np.array([[1,2,3],[4,5,6]])
print("数组形状:", arr.shape)
# 输出:(2, 3)
print("数组维度:", arr.ndim)
# 输出:2
print("数组元素总数:", arr.size)
# 输出:6
print("数组数据类型:", arr.dtype)
# 输出:int64
```

NumPy库常用内置方法

● 数组索引和切片

- 通过索引访问数组中的单个元素：array[index]
- 使用切片访问数组的子集：array[start:stop:step]
- 通过索引数组访问多个元素：array[indices]

```
array = np.array([[ 0,  1,  2,  3],[10, 11, 12, 13],[20, 21, 22, 23]])
row_indices = [0, 1, 2]
col_indices = [1, 2, 3]
mask = np.array([True, False, True])
print(array[1, 2]) # 12
print(array[1, 1:4:2]) # [11 13]
print(array[0:3:2, 1:3]) # [[ 1  2],[21 22]]
print(array[row_indices, col_indices]) # [ 1 12 23]
print(arr[mask]) # [[ 0,  1,  2,  3],[20, 21, 22, 23]]
```

注：切片返回的是原数组视图，索引数组返回的是新数组副本

NumPy库常用内置方法

● 数组形状操作

- 将数组重排为指定形状： `arr.reshape()`
- 将数组重排为指定形状，缺少的话循环使用原数组的元素来填充新的形状： `arr.resize()`
- 数组展平： `arr.ravel()` 和 `flatten()`
- 数组转置： `arr.transpose()` 和 `arr.T`
- 在指定位置插入一个新的维度： `arr.expand_dims()`
- 用于移除数组中的单维： `arr.squeeze()`

NumPy库常用内置方法

● 数组运算

- 加法: `+/np.add()`
- 减法: `-/np.subtract()`
- 点积: `@/np.dot()`
- 对应元素乘积: `*/np.multiply()`
- 幂乘: `**/np.power()`
- 矩阵乘法: `np.matmul()`
- 绝对值: `np.abs()`

广播机制: 自动扩展形状不同的数组, 使它们在元素级别进行兼容运算

```
import numpy as np
a = np.array([1, 2, 3])
b = np.array([10, 20, 30])
np.add(a, b) # 加法 [11, 22, 33]
np.subtract(a, b) # 减法 [-9, -18, -27]
np.multiply(a, 2) # 对应元素乘积 [2, 4, 6]
np.dot(a, b) # 点积 140
np.power(a, 2) # 幂乘 [1, 4, 9]
np.matmul(a, b) # 矩阵乘法 140
np.abs(np.subtract(a, b)) # 绝对值 [9, 18, 27]
np.add(a, 2) # 广播机制 [9, 18, 27]
```

NumPy库常用内置方法

● 数组的数学函数（统计方法）

- 所有元素和： `np.sum()`
- 排序： `np.sort()`
- 去重并排序： `np.unique()`
- 平均值： `np.mean()`
- 中位数： `np.median()`
- 最大最小值： `np.max()`和`min()`
- 判断是否有NaN值： `np.isnan()`
- NaN值替换为指定的值：
`np.nan_to_num()`

```
import numpy as np
a = np.array([1, 4, 5.5, 3, 4])
b = np.array([3, np.nan, 1])
np.sum(a) # 求和 17.5
np.sort(a) # 排序 [1, 3, 4, 4, 5.5]
np.unique(a) # 去重排序 [1, 3, 4, 5.5]
np.mean(a) # 平均值 3.5
np.median(a) # 中位数 4.0
np.max(a) # 最大值 5.5
# 判断NaN值
np.isnan(b) # [False, True, False]
# NaN值替换为指定值
np.nan_to_num(2) # [3, 2, 1]
```

NumPy库常用内置方法

● 一维多项式类

■ `np.poly1d(c_or_r, r = False, variable = None)`

- `c_or_r`: 若`r = False`, 此项为多项式系数, 以幂次递减;
若`r = True`, 此项为多项式的根
- `variable`: 更改多项式变量为`variable`

■ 构建多项式`p`后,

- 计算多项式在某个`x`处的值: `p(x)`
- 求多项式的根: `p.r`
- 显示多项式的系数: `p.c`
- 显示多项式的阶数: `p.order`
- 返回第`k`次幂的系数: `p[k]`

NumPy库常用内置方法

● 一维多项式类

```
import numpy as np
# 构造多项式  $x^2+2x+3$ 
coeff = np.array([1, 2, 3]) # 多项式系数
p = np.poly1d(coeff, False, 'x') # 构造多项式
p(0.5) # 给定 $x=0.5$ , 求多项式的值 4.5
p.r() # 求多项式的根  $[-1.+1.41421356j -1.-1.41421356j]$ 
p.c() # 显示多项式的系数  $[1, 2, 3]$ 
p.order() # 显示多项式的阶数 2
p[0], p[1], p[2] # 显示第 $k$ 次幂的系数 3, 2, 1

p_1 = np.poly1d(coeff, False, 'z') # 构造以 $z$ 为变量的多项式  $z^2+2z+3$ 
p_1 = np.poly1d(coeff, True) # 构造以 $[1, 2, 3]$ 为根的多项式  $x^3-6x^2+11x-6$ 
```

NumPy库常用内置方法

● 线性代数

- 计算输入矩阵的行列式: `np.linalg.det()`
- 返回计算矩阵的乘法逆矩阵: `np.linalg.inv()`
- 返回方阵的特征值和特征向量: `np.linalg.eig()`
- 矩阵SVD分解: `np.linalg.svd()`

```
import numpy as np
from numpy import linalg as LA
a = np.diag((1,2,3)) # 定义对角矩阵 [[1, 0, 0], [0, 2, 0], [0, 0, 3]]
LA.det(a) # 计算行列式 6.0
LA.inv(a) # 计算逆矩阵 [[1, 0, 0], [0, 0.5, 0], [0, 0, 0.3333333]]
w, v = LA.eig(a) # 求特征值和特征向量 [1, 2, 3] [[1, 0, 0], [0, 1, 0], [0, 0, 1]]
U, s, Vt = LA.svd(a) # SVD分解, 得到U矩阵 [[0, 0, 1], [0, 1, 0], [1, 0, 0]]、奇异值 [3, 2, 1]与V矩阵 [[0, 0, 1], [0, 1, 0], [1, 0, 0]]
```

2.2 科学计算库: SciPy

SciPy库

● SciPy库简介

- Python中的科学计算库，全称Scientific Python
- SciPy依赖于NumPy，提供高效的数值计算、优化、信号处理、统计等工具
- SciPy的主要功能包括：
 - 优化求解：最小二乘拟合`leastsq()`、多项式拟合`polyfit()`、多项式求值`polyval()`
 - 线性代数运算：计算行列式`linalg.det()`、计算逆矩阵`linalg.inv()`、奇异值分解`linalg.svd()`等
 - 图像处理：读取图像`ndimage.imread()`、图像旋转`ndimage.rotate()`、图像缩放`ndimage.zoom()`等

SciPy库优化求解

● 最小二乘拟合函数: `leastsq()`

■ 对于一组实验观测数据，寻求一个近似函数 $y = f(x)$ ，使其在**最小二乘原则**（ $J = \sum_{i=1}^n \delta_i^2 = \sum_{i=1}^n [f(x_i) - y_i]^2$ ）下与已知的 n 个离散数据点最接近，即实现拟合误差最小

```
from scipy.optimize import leastsq  
x = leastsq(func, x0, args)
```

■ 对于最小二乘拟合函数`x = leastsq(func, x0, args)`，1) 输入参数`func`为偏差函数，用于计算拟合数据与实验数据之间的误差；`x0`为待拟合参数的初始值数组；`args`为需要拟合的实验数据数组；2) 返回值`x`求解的拟合参数数组

SciPy库优化求解

● 多项式拟合函数: polyfit()

■ 对于一组实验观测数据，其拟合模型为 $f(x) = a_n x^n + a_{n-1} x^{n-1} \cdots a_1 x + a_0$ ，寻找一组多项式系数 a_n ，使得 $f(x)$ 能够较好拟合原始数据，使整体拟合误差较小

```
import numpy as np  
p = np.polyfit(x, y, n)
```

■ 对于多项式拟合函数 $p = \text{polyfit}(x, y, n)$ ，1) 输入参数 x ， y 为要拟合的数据，为序列类型（列表，元组或数组）； n 为拟合多项式的次数；2) 返回值 p 拟合多项式系数数组

SciPy库优化求解

● 多项式求值函数: `polyval()`

- 用于在求得拟合多项式后，计算多项式在任意x处的值y

```
import numpy as np  
y = np.polyval(p, x)
```

- 对于多项式求值函数 $y = \text{polyval}(p, x)$ ，1) 输入参数p为多项式拟合函数`polyval()`返回的拟合多项式系数；x为任意一个坐标值，可以是一个值，也可以是一组存储于列表、元组或数组中的值；2) 返回值y为该多项式在x处的值

2.3 数据分析库: Pandas

Pandas库

● Pandas库简介

■ Pandas 是一个基于 Python 的数据分析与处理库，主要用于结构化数据（如表格、时间序列）的操作

■ Pandas常用功能：

- 数据读取与写入
- 数据清洗
- 数据选择与筛选
- 数据分组
- 数据聚合与统计（平均值、总和、计数等）
- 时间序列处理

Pandas一维数据操作

● 数据创建

- 使用pd.Series可以创建一维数据
- **默认索引**为从0开始的自然数序列

```
import pandas as pd
s_1 = pd.Series([10, 20, 30, 40, 50])
# 使用默认索引的一维数据s_1
# 0    10
# 1    20
# 2    30
# 3    40
# 4    50
# dtype: int64
```

	s_1
0	10
1	20
2	30
3	40
4	50

Pandas一维数据操作

● 数据创建

- 使用pd.Series可以创建一维数据
- 通过index参数可以自定义索引

```
s_2 = pd.Series([100, 200, 300, 400, 500], index=['a', 'b', 'c', 'd', 'e'])  
# 使用自定义索引的一维数据s_2  
# a    100  
# b    200  
# c    300  
# d    400  
# e    500  
# dtype: int64
```

	s_2
'a'	10
'b'	20
'c'	30
'd'	40
'e'	50

Pandas一维数据操作

● 数据访问

■ 访问元素：与列表和字典类似

- 若没有自定义索引，则通过下标访问元素
- 若有自定义索引，既可以通过下标访问元素，也可以通过自定义索引访问元素

	s_1		s_2
0	10	'a'	100
1	20	'b'	200
2	30	'c'	300
3	40	'd'	400
4	50	'e'	500

```
import pandas as pd
s_1 = pd.Series([10, 20, 30, 40, 50])
s_1[1] # 通过下标访问元素 200

s_2 = pd.Series([100, 200, 300, 400, 500], index=['a', 'b', 'c', 'd', 'e'])
s_2['b'] # 通过自定义索引访问元素 200
```

Pandas一维数据操作

● 数据切片

■ 切片操作：与列表相似

```
import pandas as pd
s_1[1:3] # 不包含下标为3的元素
# b    200
# c    300
# dtype: int64

s_2['b':'d'] # 包含索引为'd'的元素
# b    200
# c    300
# d    400
# dtype: int64
```

	s_1		s_2
0	10	'a'	100
1	20	'b'	200
2	30	'c'	300
3	40	'd'	400
4	50	'e'	500

Pandas一维数据操作

● 数据统计

- 求和: `sum()`
- 计算平均值: `mean()`
- 计算最大值: `max()`
- 计算最小值: `min()`
- 计算标准差: `std()`

	s_1		s_2
0	10	'a'	100
1	20	'b'	200
2	30	'c'	300
3	40	'd'	400
4	50	'e'	500

```
import pandas as pd
s_2.sum() # 求和 1500
s_2.mean() # 计算平均值 300.0
s_2.max() # 计算最大值 500
s_2.min() # 计算最小值 100
s_2.std() # 计算标准差 158.11388300841898
```

Pandas二维数据操作

● Pandas二维数据

■ DataFrame: DataFrame 是 Pandas 的核心数据结构，用于表示二维表格数据，类似于 Excel 表格或者数据库中的表格

■ DataFrame主要特点：

- 每一列可以是不同的数据类型（数字、字符串等）
- 包含行索引和列标签，便于定位和操作数据
- 支持高效的数据读取、筛选、聚合等功能

Pandas二维数据操作

● 数据创建

■ 创建DataFrame

```
import pandas as pd
data = {
    '姓名': ['张三', '李四', '王五'],
    '年龄': [23, 25, 22],
    '成绩': [88, 92, 85]}
df = pd.DataFrame(data)
# 二维数据df
#   姓名  年龄  成绩
# 0 张三   23   88
# 1 李四   25   92
# 2 王五   22   85
```

df			
	姓名	年龄	成绩
0	张三	23	88
1	李四	25	92
2	王五	22	85

Pandas二维数据操作

● 数据访问

- 可通过列索引访问
- 访问一列时返回Series
- 访问多列时返回DataFrame

df			
	姓名	年龄	成绩
0	张三	23	88
1	李四	25	92
2	王五	22	85

```
import pandas as pd
df['姓名'] # 访问一列
# 0 张三
# 1 李四
# 2 王五
# Name: 姓名, dtype: object
```

```
import pandas as pd
df[['姓名', '成绩']] # 访问多列
# 姓名 成绩
# 0 张三 88
# 1 李四 92
# 2 王五 85
```

Pandas二维数据操作

● 数据访问

■ 按索引访问行列数据

➤ 按标签进行数据访问: `df.loc[label]`

➤ 按下标进行数据访问: `df.iloc[index]`

df			
	姓名	年龄	成绩
0	张三	23	88
1	李四	25	92
2	王五	22	85

```
import pandas as pd
df.loc[:, '成绩'] # 获取成绩列
# 0    88
# 1    92
# 2    85
# Name: 成绩, dtype: int64
```

```
import pandas as pd
df.iloc[0] # 获取第0行
# 姓名 张三
# 年龄  23
# 成绩  88
# Name: 0, dtype: object
```

Pandas二维数据操作

● 数据访问

■ 按索引访问具体单元格数据

- 按标签进行数据访问: `df.at[label]`
- 按下标进行索引: `df.iat[index]`

df			
	姓名	年龄	成绩
0	张三	23	88
1	李四	25	92
2	王五	22	85

```
import pandas as pd
df.at[1, '成绩'] # 访问第1行"成绩"列的值 92
df.iat[1, 2] # 访问第1行第2列的值 92
```

Pandas二维数据操作

● 数据筛选

■ 使用df[条件语句]进行筛选

■ "&": 逻辑且

■ "|": 逻辑或

df			
	姓名	年龄	成绩
0	张三	23	88
1	李四	25	92
2	王五	22	85

```
import pandas as pd
# 筛选年龄大于23的人
df[df['年龄'] > 23]
# 姓名 年龄 成绩
# 1 李四 25 92
```

```
import pandas as pd
# 筛选年龄大于23且成绩大于90的人
df[(df['年龄'] > 23) & (df['成绩'] > 90)]
# 姓名 年龄 成绩
#1 李四 25 92
```

Pandas二维数据操作

● 数据添加

■ 添加新列: `df[column_name] = [...]`

■ 添加新行: `df.loc[len(df)] = [...]`

df				
	姓名	年龄	成绩	等级
0	张三	23	88	B
1	李四	25	92	A
2	王五	22	85	B
3	小明	20	95	A

```
import pandas as pd
# 添加新列
df['等级'] = ['B', 'A', 'B']
# 姓名 年龄 成绩 等级
# 0 张三 23 88 B
# 1 李四 25 92 A
# 2 王五 22 85 B
```

```
import pandas as pd
# 添加新行
df.loc[len(df)] = ['小明', 20, 95, 'A']
# 姓名 年龄 成绩 等级
# 0 张三 23 88 B
# 1 李四 25 92 A
# 2 王五 22 85 B
# 3 小明 20 95 A
```

Pandas二维数据操作

● 数据修改

■ 直接对df.at[index]赋值，可以修改单单元格数据

```
import pandas as pd
df.at[0, '等级'] = 'C' # 修改第0行、"等级"列的值
# 姓名 年龄 成绩 等级
# 0 张三 23 88 C
# 1 李四 25 92 A
# 2 王五 22 85 B
# 3 小明 20 95 A
```

df				
	姓名	年龄	成绩	等级
0	张三	23	88	C
1	李四	25	92	A
2	王五	22	85	B
3	小明	20	95	A

Pandas二维数据操作

● 数据删除

■ 使用`df.drop(index, inplace)`可以删除整行或整列

■ `inplace=True`表示直接对原始表格进行修改; `inplace=False`表示生成并返回新的副本

```
import pandas as pd
# 直接在原始表格中删除等级列
df.drop(columns='等级', inplace=True)
# 姓名 年龄 成绩
# 0 张三 23 88
# 1 李四 25 92
# 2 王五 22 85
# 3 小明 20 95
```

df			
	姓名	年龄	成绩
0	张三	23	88
1	李四	25	92
2	王五	22	85
3	小明	20	95

Pandas二维数据操作

● 数据统计

- 与一维数据统计函数类似
- 求和: `sum()`
- 计算平均值: `mean()`
- 计算最大值: `max()`
- 计算最小值: `min()`
- 计算标准差: `std()`

df			
	姓名	年龄	成绩
0	张三	23	88
1	李四	25	92
2	王五	22	85
3	小明	20	95

```
import pandas as pd
df['成绩'].mean() # 计算成绩平均值 90
df['年龄'].max() # 计算年龄最大值 25
df['成绩'].std() # 计算成绩标准差 3.80788655
```


Pandas二维数据操作

● 数据排序

- 使用`df.sort_values(by, ascending)`进行排序
- `by`指定要排序的列
- `ascending = True`为升序；`ascending = False`为降序

```
import pandas as pd
# 按成绩降序排列
df.sort_values(by='成绩', ascending=False)
#   姓名  年龄  成绩
# 3  小明   20   95
# 1  李四   25   92
# 0  张三   23   88
# 2  王五   22   85
```

df			
	姓名	年龄	成绩
0	张三	23	88
1	李四	25	92
2	王五	22	85
3	小明	20	95

Pandas数据清洗

● 数据清洗定义及操作

■ 是对一些没有用的数据进行处理的过程，常见类型有：
数据缺失、数据格式错误、错误数据以及重复数据

■ 空值类型包括：np.nan (Not a Number)、None、
pd.NaT (时间格式的空值)

■ 常用操作：

- 判断单元格是否为空：isnull()
- 删除包含空字段的行：dropna()
- 替换空字段：fillna()
- 用缺失值的前一个值填充：pad()
- 用缺失值的后一个值填充：bfill()

Pandas文件操作

● 文件的导入与导出

■ 导入文件：pandas支持导入CSV文件、Excel文件、分隔的文本文件、SQL表/数据库等

```
import pandas as pd  
df_excel = pd.read_excel('name.xlsx') # 导入 Excel 文件  
df_csv = pd.read_csv('name.csv') # 导入 CSV 文件
```

■ 导出文件：经过处理后的数据支持导出CSV文件、Excel文件、JSON文件、SQL表

```
import pandas as pd  
df_excel.to_excel('name.xlsx') # 导出 Excel 文件  
df_csv.to_csv('name.csv') # 导出 CSV 文件
```

Pandas应用实例

● 数据介绍

- 该应用实例采用NBA球星作为实际背景，主要包含：姓名、球队名称、号码、位置、年龄、身高、体重、毕业院校、薪水等基本信息
- 姓名、球队名称、位置、毕业院校为字符串形式，其余信息为数字形式，其中部分球员的毕业院校和薪水存在空值
- 数据集文件名称为NBA.csv，是一个CSV格式文件，原始文件共9个字段，457条信息，可由Pandas库直接进行处理

Pandas应用实例

● 数据导入

- 导入NBA.csv文件中的所有数据，构建二维数据表

```
import pandas as pd
df = pd.read_csv('NBA.csv') # 导入NBA.csv文件
print("df:\n", df) # 打印前5条与后5条数据
```

```
df:
   姓名  球队名称  号码  ...  体重  毕业院校  薪水
0  Avery Bradley  Boston Celtics  0  ...  180  Texas  7730337.0
1    Jae Crowder  Boston Celtics  99  ...  235  Marquette  6796117.0
2   John Holland  Boston Celtics  30  ...  205  Boston University  NaN
3    R.J. Hunter  Boston Celtics  28  ...  185  Georgia State  1148640.0
4   Jonas Jerebko  Boston Celtics  8  ...  231  NaN  5000000.0
..  ...  ...  ..  ...  ...  ...  ...
452   Trey Lyles  Utah Jazz  41  ...  234  Kentucky  2239800.0
453  Shelvin Mack  Utah Jazz  8  ...  203  Butler  2433333.0
454   Raul Neto  Utah Jazz  25  ...  179  NaN  900000.0
455  Tibor Pleiss  Utah Jazz  21  ...  256  NaN  2900000.0
456   Jeff Withey  Utah Jazz  24  ...  231  Kansas  947276.0

[457 rows x 9 columns]
```

Pandas应用实例

● 数据访问

- 按下列要求对数据进行访问：
 - 访问所有球员"姓名"信息

```
import pandas as pd
df_name = df['姓名'] # 访问姓名信息
print("df_name:\n", df_name)
```

```
df_name:
0      Avery Bradley
1      Jae Crowder
2      John Holland
3      R.J. Hunter
4      Jonas Jerebko
...
452     Trey Lyles
453     Shelvin Mack
454     Raul Neto
455     Tibor Pleiss
456     Jeff Withey
Name: 姓名, Length: 457, dtype: object
```

Pandas应用实例

● 数据访问

■ 按下列要求对数据进行访问：

➤ 同时访问所有球员"身高"及"体重"信息

```
import pandas as pd
df_hw = df[['身高', '体重']] # 访问身高、
    体重信息
print("df_hw:\n", df_hw)
```

```
df_hw:
      身高  体重
0      6.2  180
1      6.6  235
2      6.5  205
3      6.5  185
4      6.1  231
..     ...   ...
452    6.1  234
453    6.3  203
454    6.1  179
455    7.3  256
456    7.0  231

[457 rows x 2 columns]
```

Pandas应用实例

● 数据访问

■ 按下列要求对数据进行访问：

➤ 访问前五名球员的"毕业院校"

```
import pandas as pd
df_college_5 = df.loc[0:4, '毕业院校'] # 访问前五名球员毕业院校信息
print("df_college_5:\n", df_college_5)
```

```
df_college_5:
0          Texas
1      Marquette
2  Boston University
3      Georgia State
4             NaN
Name: 毕业院校, dtype: object
```


Pandas应用实例

● 数据访问

■ 按下列要求对数据进行访问：

➤ 访问最后一名球员的所有信息

```
import pandas as pd
df_player_457 = df.iloc[456] #访问最后一名球员所有信息
print("df_player_457:\n", df_player_457)
```

```
df_player_457:
  姓名      Jeff Withey
  球队名称    Utah Jazz
  号码          24
  位置          C
  年龄          26
  身高          7.0
  体重          231
  毕业院校      Kansas
  薪水      947276.0
Name: 456, dtype: object
```

Pandas应用实例

● 数据筛选

■ 筛选年龄<25岁且薪水>10000000的球员的所有信息

```
import pandas as pd
df_yr = df[(df['年龄'] < 25) & (df['薪水'] >
10000000)] # 筛选年龄<25且薪水
>10000000的球员信息
print("df_yr:\n", df_yr)
```

```
df_yr:
      姓名      球队名称  ...      毕业院校      薪水
127  Brandon Knight      Phoenix Suns  ...      Kentucky  13500000.0
168   Kyrie Irving      Cleveland Cavaliers  ...      Duke  16407501.0
189   Tobias Harris      Detroit Pistons  ...      Tennessee  16000000.0
219  Khris Middleton      Milwaukee Bucks  ...      Texas A&M  14700000.0
301   Kawhi Leonard      San Antonio Spurs  ...  San Diego State  16407500.0
418   Enes Kanter      Oklahoma City Thunder  ...      Kentucky  16407500.0
446  Derrick Favors      Utah Jazz  ...      Georgia Tech  12000000.0

[7 rows x 9 columns]
```

Pandas应用实例

● 数据修改

■ 修改原数据的列索引为英文形式

```
import pandas as pd
df.columns = ['Name', 'Team',
'Number', 'Position', 'Age', 'Height',
'Weight', 'College', 'Salary'] # 修改数
据列索引
print("df:\n", df)
```

```
df:
      Name      Team  ...      College      Salary
0  Avery Bradley  Boston Celtics  ...      Texas  7730337.0
1    Jae Crowder  Boston Celtics  ...  Marquette  6796117.0
2   John Holland  Boston Celtics  ...  Boston University      NaN
3    R.J. Hunter  Boston Celtics  ...   Georgia State  1148640.0
4   Jonas Jerebko  Boston Celtics  ...      NaN  5000000.0
..      ...      ...  ...      ...      ...
452   Trey Lyles      Utah Jazz  ...   Kentucky  2239800.0
453  Shelvin Mack      Utah Jazz  ...    Butler  2433333.0
454    Raul Neto      Utah Jazz  ...      NaN    900000.0
455  Tibor Pleiss      Utah Jazz  ...      NaN  2900000.0
456   Jeff Withey      Utah Jazz  ...   Kansas    947276.0

[457 rows x 9 columns]
```

Pandas应用实例

● 数据统计

- 对所有球员的薪水信息进行统计，包括总和、平均值、最大值、最小值以及标准差

```
import pandas as pd
print(f"Salary_sum: {df['Salary'].sum()}") # 求和
print(f"Salary_mean: {df['Salary'].mean()}") # 平均值
print(f"Salary_max: {df['Salary'].max()}") # 最大值
print(f"Salary_min: {df['Salary'].min()}") # 最小值
print(f"Salary_std: {df['Salary'].std()}") # 标准差
```

```
Salary_sum: 2159837111.0
Salary_mean: 4842684.105381166
Salary_max: 25000000.0
Salary_min: 30888.0
Salary_std: 5229237.598910348
```

Pandas应用实例

● 数据排序

- 对所有球员的薪水进行降序排序，并直接将排序结果更新至原始数据中

```
import pandas as pd
df.sort_values(by='Salary', ascending=False, inplace=True)
# 将薪水按照降序排序，并更新原始数据
print("df:\n", df)
```

```
df:
      Name      Team ...      College      Salary
109  Kobe Bryant  Los Angeles Lakers ...      NaN  25000000.0
169  LeBron James  Cleveland Cavaliers ...      NaN  22970500.0
33   Carmelo Anthony  New York Knicks ...      Syracuse  22875000.0
251  Dwight Howard  Houston Rockets ...      NaN  22359364.0
339  Chris Bosh     Miami Heat ...      Georgia Tech  22192730.0
..      ...      ...      ...      ...      ...
273  Alex Stepheson  Memphis Grizzlies ...      USC      NaN
350  Briante Weber   Miami Heat ...      Virginia Commonwealth  NaN
353  Dorell Wright   Miami Heat ...      NaN      NaN
397  Axel Toupane    Denver Nuggets ...      NaN      NaN
409  Greg Smith     Minnesota Timberwolves ...      Fresno State  NaN

[457 rows x 9 columns]
```

Pandas应用实例

● 数据清洗

■ 确认包含空值的球员信息或信息类别

```
import pandas as pd
rows_with_null = df[df.isnull().any(axis=1)] # 找到包含空值的球员
print("df_null_rows:\n", rows_with_null)
columns_with_null = df.columns[df.isnull().any()] # 找到包含空值的信息
print("df_null_columns:\n", columns_with_null)
```

```
df_null_rows:
   Name      Team ...      College  Salary
109  Kobe Bryant  Los Angeles Lakers ...      NaN  25000000.0
169  LeBron James  Cleveland Cavaliers ...      NaN  22970500.0
251  Dwight Howard   Houston Rockets ...      NaN  22359364.0
265   Marc Gasol   Memphis Grizzlies ...      NaN  19688000.0
341  Goran Dragic    Miami Heat ...      NaN  14783000.0
..      ...      ...      ...      ...
273 Alex Stepheson  Memphis Grizzlies ...      USC      NaN
350  Briante Weber    Miami Heat ...  Virginia Commonwealth  NaN
353  Dorell Wright    Miami Heat ...      NaN      NaN
397  Axel Toupane   Denver Nuggets ...      NaN      NaN
409   Greg Smith  Minnesota Timberwolves ...  Fresno State  NaN

[93 rows x 9 columns]
```

```
df_null_columns:
Index(['College', 'Salary'], dtype='object')
```

Pandas应用实例

● 数据清洗

■ 确认包含空值的球员信息及类别

```
import pandas as pd
null_indices = df.isnull().stack()
null_indices_list = []
# 遍历包含空值的行列索引
for row, col in null_indices[null_indices].index:
    name_value = df.iloc[row]['Name']
    null_indices_list.append([name_value, col])
print("df_null:")
for i in range(10):
    print(null_indices_list[i])
```

```
df_null:
['Jae Crowder', 'College']
['Lavoy Allen', 'College']
['Ramon Sessions', 'College']
['Kent Bazemore', 'College']
['Walter Tavares', 'College']
['Dwight Powell', 'College']
['Cole Aldrich', 'College']
['Brandon Rush', 'College']
['Josh Huestis', 'College']
['Josh Smith', 'College']
```

Pandas应用实例

● 数据清洗

- 删除毕业院校为空值的球员信息，并用0填充薪水空值

```
import pandas as pd
df.dropna(subset=['College'], inplace=True) # 删除毕业院校为空的球员，共84条
df.fillna(0, inplace=True) # 用0填充薪水为空的球员的薪水，共11条
print("df:\n", df)
```

```
df:
      Name      Team ...      College      Salary
33  Carmelo Anthony  New York Knicks ...      Syracuse  22875000.0
339  Chris Bosh      Miami Heat ...      Georgia Tech  22192730.0
100  Chris Paul    Los Angeles Clippers ...      Wake Forest  21468695.0
414  Kevin Durant  Oklahoma City Thunder ...      Texas  20158622.0
164  Derrick Rose      Chicago Bulls ...      Memphis  20093064.0
..      ...      ...      ...      ...
269  Ray McCallum  Memphis Grizzlies ...      Detroit      0.0
270  Xavier Munford  Memphis Grizzlies ...      Rhode Island      0.0
273  Alex Stepheson  Memphis Grizzlies ...      USC      0.0
350  Briante Weber      Miami Heat ...  Virginia Commonwealth      0.0
409  Greg Smith    Minnesota Timberwolves ...      Fresno State      0.0

[373 rows x 9 columns]
```


Pandas应用实例

● 数据导出

- 将处理后的数据，导出至NBA_new.csv文件中

```
import pandas as pd  
df_csv.to_csv('NBA_new.csv') # 导出CSV文件
```

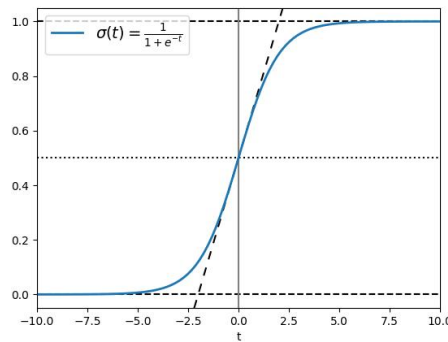
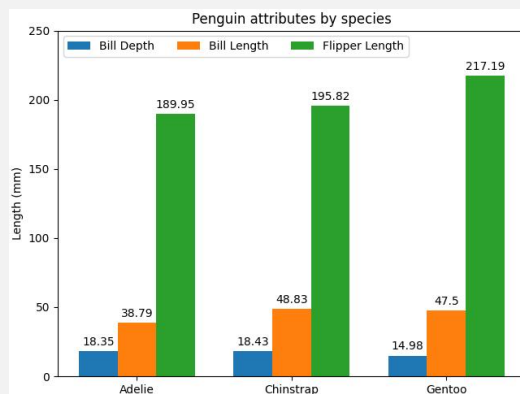
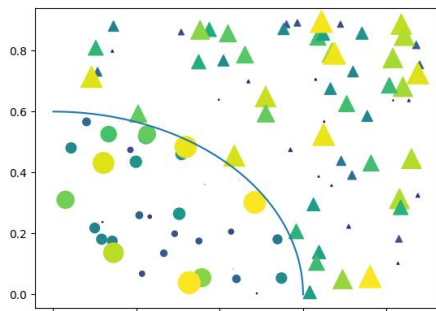
Name	Team	Number	Position	Age	Height	Weight	College	Salary
Carmelo Anthony	New York Knicks	7	SF	32	6.8	240	Syracuse	22875000
Chris Bosh	Miami Heat	1	PF	32	6.11	235	Georgia Tech	22192730
Chris Paul	Los Angeles Clippers	3	PG	31	6	175	Wake Forest	21468695
Kevin Durant	Oklahoma City Thunder	35	SF	27	6.9	240	Texas	20158622
Derrick Rose	Chicago Bulls	1	PG	27	6.3	190	Memphis	20093064
Dwyane Wade	Miami Heat	3	SG	34	6.4	220	Marquette	20000000
LaMarcus Aldridge	San Antonio Spurs	12	PF	30	6.11	240	Texas	19689000
DeAndre Jordan	Los Angeles Clippers	6	C	27	6.11	265	Texas A&M	19689000
Brook Lopez	Brooklyn Nets	11	C	28	7	275	Stanford	19689000
Kevin Love	Cleveland Cavaliers	0	PF	27	6.1	251	UCLA	19689000

2.4 绘图工具库: Matplotlib

Matplotlib库

● Matplotlib库简介

- Python 中最流行的数据可视化绘图库之一，能够以高质量生成各种静态、交互式、动画的二维图形
- 支持线图、散点图、柱状图、饼图、直方图、热力图等，几乎涵盖所有常见可视化类型



一些常见的可视化图形，从左至右依次为散点图，柱状图和线图

折线图绘制

● 折线图：plot()

- 用于绘制连续曲线或线段连接的图形，适合展示随连续变量变化的趋势

用法

```
plot(x, y, **kwargs)
```

x: 数据点的横坐标值，列表、元组或数组
y: 数据点的纵坐标值，列表、元组或数组
***kwargs*: 绘图样式的参数

- 可以通过**kwargs参数指明绘图样式

```
plot(X, Y, color, linewidth, linestyle, label, marker)
```

color指定颜色，linewidth指定线宽，linestyle指定折线形状，label指定图例的文字说明，marker指定数据点样式

折线图绘制

● 折线图-基础代码示例

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
# 在区间[0,5] 生成50个点
```

```
x = np.linspace(0, 5, 50)
```

```
y = x ** 2
```

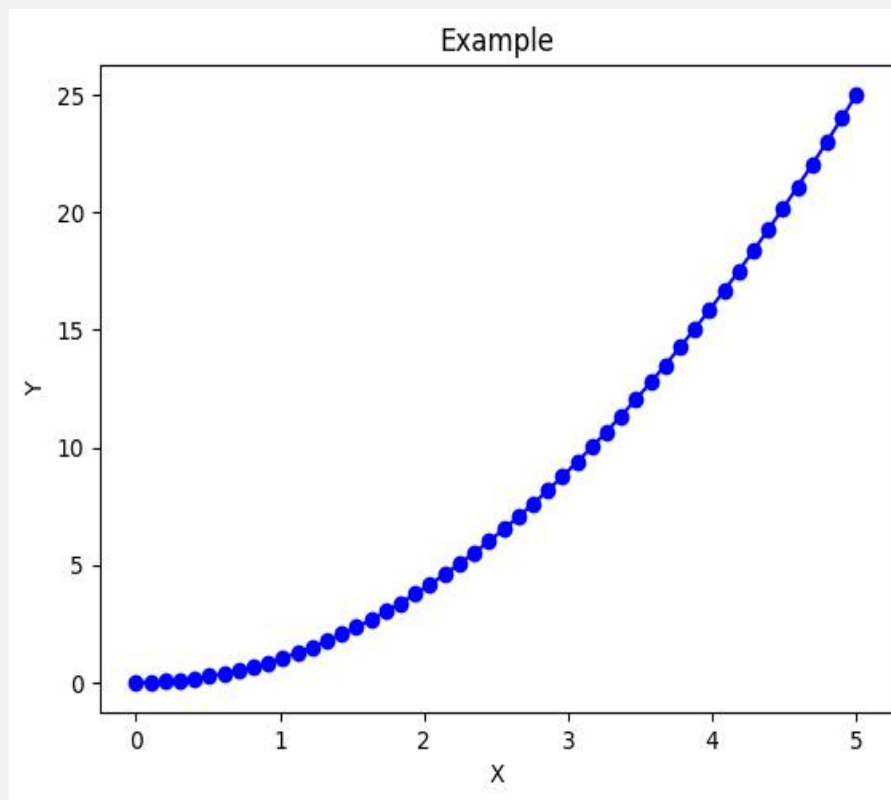
```
# 绘制蓝色实线，并在数据点标记圆圈
```

```
plt.plot(x, y, color='b', marker='o')
```

```
plt.title('Example')
```

```
plt.xlabel('X'); plt.ylabel('Y')
```

```
plt.show()
```

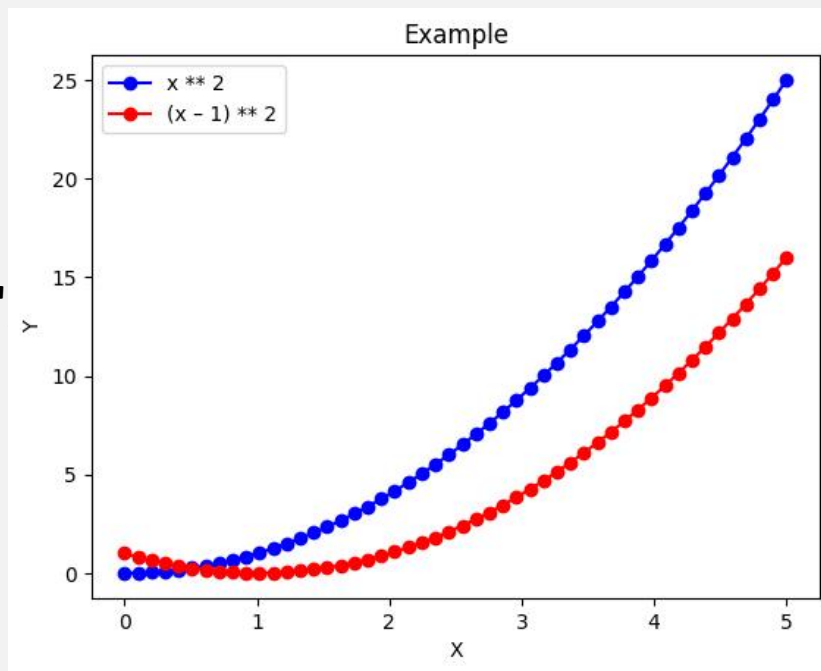


折线图绘制

● 折线图-图例(legend)使用

- 当一个绘图区域包含两条（或以上）折线时，需要添加图例(legend)，对每条折线的含义加以说明

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 5, 50)
y0 = x ** 2; y1 = (x - 1) ** 2 # 生成坐标
plt.plot(x, y0, label="x ** 2", color='b', marker='o')
plt.plot(x, y1, label="(x - 1) ** 2", color='r',
marker='o') # 绘制折线图
plt.title('Example') # 添加标题
plt.xlabel('X'); plt.ylabel('Y') # 添加标签
plt.legend() # 添加图例
plt.show()
```



散点图绘制

● 散点图：scatter()

- 用于绘制(x, y)点对，适合展示两变量之间的关系或分布

用法

```
scatter(x, y, **kwargs)
```

x: 数据点的横坐标值，列表、元组或数组
y: 数据点的纵坐标值，列表、元组或数组
***kwargs*: 绘图样式的参数

- 可以通过**kwargs参数指明绘图样式

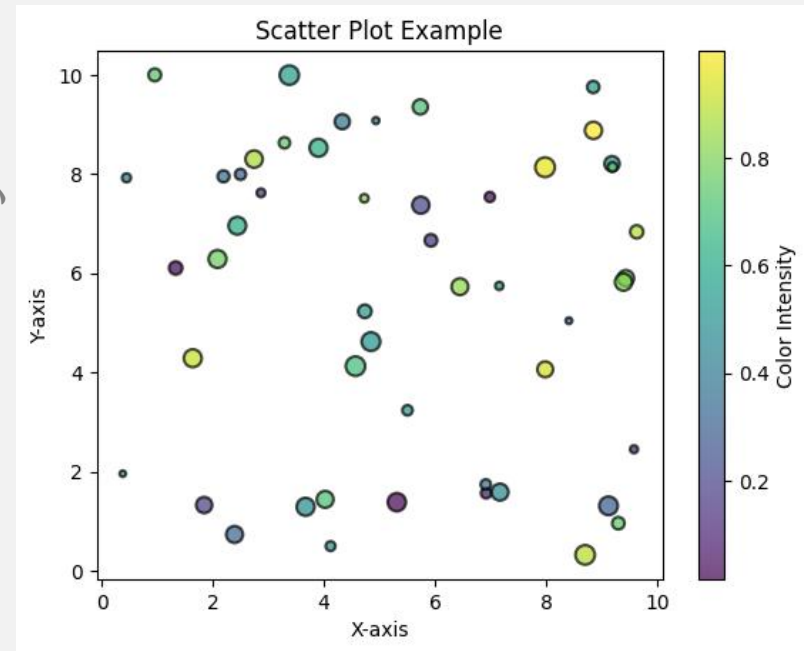
```
scatter(X, Y, s, c, marker, cmap, alpha, edgecolors, linewidths)
```

s指定散点大小，c指定散点颜色，maker指定数据点样式，cmap指定颜色映射表，alpha指定散点透明度，edgecolors指定散点边缘颜色，linewidths指定散点边缘线宽

散点图绘制

● 散点图-基础代码示例

```
import matplotlib.pyplot as plt
import numpy as np
x = np.random.rand(50) * 10
y = np.random.rand(50) * 10 # 生成坐标
sizes = np.random.randint(10, 100, 50) # 点的大小
colors = np.random.rand(50) # 点的颜色值
plt.scatter(x, y, s=sizes, c=colors, marker='o',
            cmap='viridis', alpha=0.7, edgecolors='black',
            linewidths=1.5) # 绘制散点图
plt.title("Scatter Plot Example") # 添加标题
plt.xlabel("X-axis"); plt.ylabel("Y-axis") # 添加标签
plt.colorbar(label="Color Intensity") # 显示颜色条
plt.show() # 显示图形
```

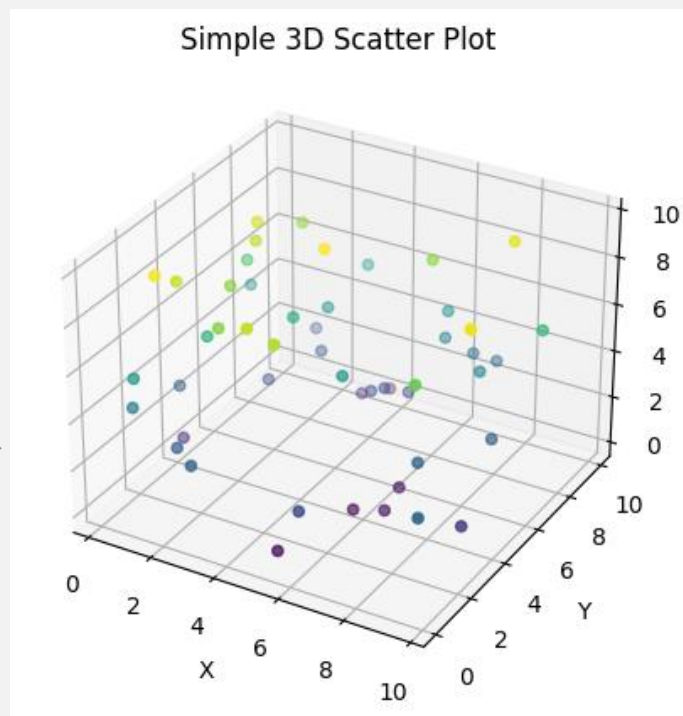


三维散点图绘制

● 三维散点图-基础代码示例

■ Matplotlib提供了Axes3D模块实现3D图形坐标创建，然后通过调用其scatter()方法进行三维散点图形绘制

```
import matplotlib.pyplot as plt
import numpy as np
x = np.random.rand(50) * 10
y = np.random.rand(50) * 10
z = np.random.rand(50) * 10
fig = plt.figure()
ax = fig.add_subplot(projection='3d')
ax.scatter(x, y, z, c=z, cmap='viridis') # 绘制三维散点图
ax.set_title("Simple 3D Scatter Plot") # 设置标题
ax.set_xlabel("X"); ax.set_ylabel("Y")
ax.set_zlabel("Z") # 设置标签
plt.show()
```



柱状图绘制

● 柱状图：bar()

- 用于绘制分类数据的柱形图，比较不同类别的数值大小

用法

```
bar(x, y, **kwargs)
```

x: 柱形中心坐标值，列表、元组或数组

y: 柱形高度，列表、元组

***kwargs*: 绘图样式的参数

- 可以通过**kwargs参数指明绘图样式

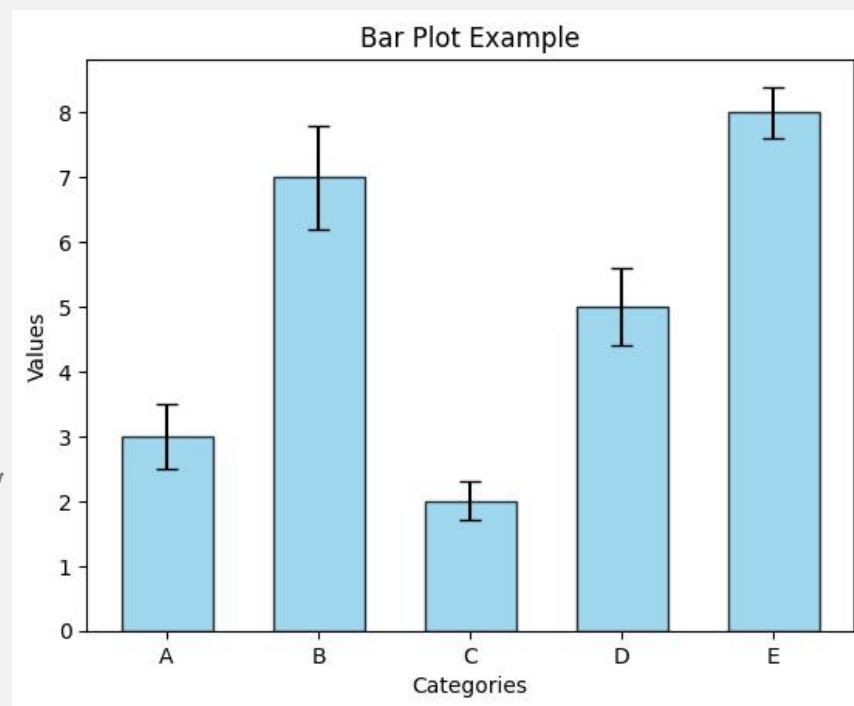
```
bar(X, Y, color, edgecolor, width, align, alpha, yerr, capsize)
```

color指定柱形颜色， edgecolors指定柱形边缘颜色， width指定柱形宽度， align指定柱形对齐方式， alpha指定柱形透明度， yerr指定误差棒值， capsize指定误差棒端点横线长度

柱状图绘制

● 柱状图-基础代码示例

```
import matplotlib.pyplot as plt
import numpy as np
categories = ['A', 'B', 'C', 'D', 'E'] # 分类标签
values = [3, 7, 2, 5, 8] # 每个分类对应的值
errors = [0.5, 0.8, 0.3, 0.6, 0.4] # 误差棒的值
plt.bar(categories, values, color='skyblue',
         edgecolor='black', width=0.6, align='center',
         alpha=0.8, yerr=errors, capsize=5) # 绘制柱状图
plt.title("Bar Plot Example") # 添加标题
plt.xlabel("Categories") # 添加X轴标签
plt.ylabel("Values") # 添加Y轴标签
plt.show()
```



饼状图绘制

● 饼状图：pie()

- 用于绘制以不同颜色呈现的扇形图，展示各部分占整体的比例关系

用法

```
pie(x, **kwargs)
```

x: 扇形数据值，列表、元组或数组
***kwargs*: 绘图样式的参数

- 可以通过**kwargs参数指明绘图样式

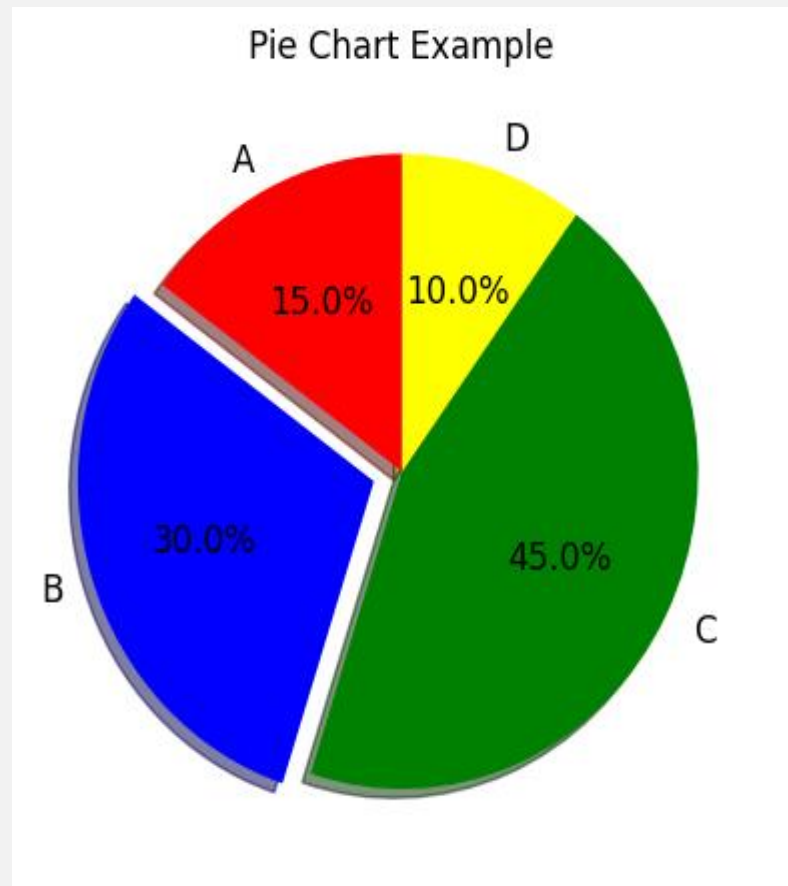
`pie(X, labels, autopct, startangle, explode, shadow, colors, textprops)`

labels指定扇形类别标签，autopct指定显示百分比格式，startangle指定起始角度，explode指定突出显示某些扇形，shadow指定阴影效果，colors指定扇形颜色，textprops指定文本属性

饼状图绘制

● 饼状图-基础代码示例

```
import matplotlib.pyplot as plt
labels = ['A', 'B', 'C', 'D'] # 分类标签
sizes = [15, 30, 45, 10] # 每个分类的值
explode = (0, 0.1, 0, 0) # 突出显示第二个切片
plt.pie(sizes, labels=labels, autopct='%1.1f%%',
startangle=90, explode=explode, shadow=True,
colors=['red', 'blue', 'green', 'yellow'],
textprops={'fontsize': 12}) # 绘制饼图
plt.title("Pie Chart Example") # 添加标题
plt.show()
```

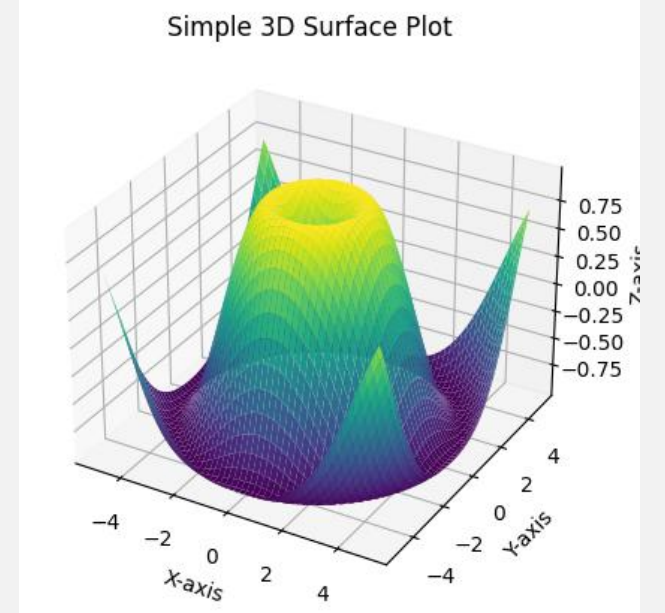


三维曲面图绘制

● 三维曲面图-基础代码示例

■ Matplotlib提供了plot_surface()方法进行三维曲面绘制

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(-5, 5, 100); y = np.linspace(-5, 5, 100)
x, y = np.meshgrid(x, y)
z = np.sin(np.sqrt(x**2 + y**2))
fig = plt.figure()
ax = fig.add_subplot(projection='3d')
ax.plot_surface(x, y, z, cmap='viridis') # 绘制三维曲面图
ax.set_title("Simple 3D Surface Plot") # 添加标题
ax.set_xlabel("X-axis"), ax.set_ylabel("Y-axis"), ax.set_zlabel("Z-axis") # 添加标签
plt.show()
```



多个子图绘制

● 子图与布局: `subplots()`

- 用于在一个Figure中展示多个子图 (`subplot`)

用法

`subplots(nrows, ncols, **kwargs)`

nrows: 子图的行数

ncols: 子图的列数

- 可以通过`**kwargs`参数指明绘图样式

`fig, axes = subplots(nrows, ncols, figsize, sharex, sharey)`

参数值: `figsize`指定整个画布的大小, `sharex`和`sharey`指定所有子图是否共享X轴或Y轴;

返回值: `fig`为整个画布, 用于调整画布属性; `axes`为子图对象数组, 对于单个子图时是单个对象, 而多个子图时是一个子图坐标二维数组

多个子图绘制

● 多个子图绘制-基础代码示例

```
import matplotlib.pyplot as plt
import numpy as np
x = np.linspace(0, 10, 100); y1 = np.sin(x); y2 = np.cos(x)
fig, axes = plt.subplots(nrows=2, ncols=1, figsize=(6, 8),
sharex=True) # 创建画布和子图
axes[0].plot(x, y1, color='blue', label='sin(x) ') # 绘制子图1
axes[0].set_title("Sine Function");
axes[0].legend(); axes[0].grid(True)
axes[1].plot(x, y2, color='red', label='cos(x) ') # 绘制子图2
axes[1].set_title("Cosine Function")
axes[1].set_xlabel("x")
axes[1].legend(); axes[1].grid(True)
plt.tight_layout() # 调整子图之间的间距
plt.show()
```

